VASTRANGAM

Vastrangam BOS

Business Operating System — one business, one brain.

21 modules · 98 apps · 16 working today · compiled 2026-08-15

What this document is

Two documents in one, because they answer two different questions and people ask both.

Part One — The System is the reader's tour: what Vastrangam BOS is, how one garment moves through it, every module and every app, and the rules that hold everywhere.

Part Two — The Plan of Action is the builder's document: the one law, the data core, the module-to-module wiring, five end-to-end flows, all 21 modules in build order with a diagram each, and what "done" means.

Both are generated from `brand/site/modules.js`, the one canonical list. Neither this page nor either part contains a module count, an app name or an app order typed by hand — which is why they cannot disagree with each other or with the software.

Where the build actually stands

16 of 98 apps are working today. Each one is a real single-file application that carries its own self-tests and passes a click-through audit in both editions. Every other app in this document is marked **designed, not yet built**, and is described as a specification rather than as something you can open. Nothing here is described as finished that is not.

Companies and channels — the short answer

The system is not limited to three companies, and never was.

A company is a row. A channel — a marketplace account, the D2C site, a POS counter, a B2B desk, an export buyer — is also a row. Every business record carries the company it belongs to; every sale also carries the channel it came through. **Ten companies with ten channels each is the same tables and the same code as three companies and seven marketplaces.**

	Today's data	The design
Companies	3	a table, no ceiling
Channels per company	7 marketplaces + D2C, B2B, export, POS	a table, no ceiling
Stock	one number per SKU	one number per SKU — never per channel
Books	one ledger per company	one ledger per company
Group	sum minus inter-company trade	sum minus inter-company trade

This is checked rather than claimed. `core/tests/core.test.js` builds ten companies with ten channels each — a hundred channels — posts an order down every one plus ten inter-company sales, and asserts that every company's books balance, that no journal line anywhere points at another company's account, and that the group figure is the plain sum **minus** inter-company trade: ₹2,10,500 gross, ₹50,000 eliminated, ₹1,60,500 group. The same builder is then called for eleven companies and eleven channels with nothing in the code changed.

The reporting side behaves the same way. **Vastrangam_BOS_Data_Studio.html** reads your own sale, return and karigar workbooks in the browser — no upload, no account, no internet — and emits one pair of quantity columns per company **found in the sheets**. A fourth company is a new sheet in the workbook, not a new version of the software.

The honesty rules this document is written under

1. Nothing is described as finished that is not. Every app is marked working today or designed.
2. No count is typed from memory. Modules, apps and build state are read from the canonical list.
3. No figure is invented. Where a rate or a price is missing, the tool posts zero and names the item rather than guessing — a guessed stitching rate is a wrong payment to a real person.
4. Where something could not be verified, it says so instead of implying it was.

PART ONE — THE SYSTEM

The Business Operating System for Vastrangam Group: 21 modules and 98 apps over one shared data core.

This file is the whole system in plain text — every module, every app, and what each one reads and writes. It is generated from `brand/site/modules.js`, the same file the website and every PDF read, so nothing here can disagree with them. The counts below are not typed in; they are counted from that file each time this page is built.

Modules	21, built in dependency order — a module is only built once everything it needs exists
Apps	98
Working today	16
Still to build	82
Companies	Vastrangam (invoices VS) · Ethnic Fashion trading as Go4Fashion (invoices EF, SKUs GF) · Adini Couture (invoices AC)
Shared data core	Company · Item/SKU · Party · Stock · Ledger/Voucher · Order
Key difference	Not a suite of integrated apps. One application over one database, so there is no sync step and no second copy of any master record
Compliance	Double-entry books with CGST/SGST/IGST, TDS, TCS, input credit on **accepted** goods, GSTR-1 and GSTR-3B, filed per registration
Channels	Amazon, Flipkart, Myntra, Meesho, Ajio, Nykaa, JioMart, plus your own storefront, the Surat counter, boutique wholesale and export
Security	Row-level isolation per company; outside services connect with scoped, revocable keys — **never account passwords**
Deployment	Hosted, or single-file apps that run by double-clicking with no install and no internet

The one idea

Every module reads and writes the **same six records**. That is the physical reason a single goods receipt can touch stock, the books, quality and sourcing in the same instant.

```
                ┌────────────────────────────────────────┐
                │           UNIFIED DATA CORE            │
                │  Company · Item/SKU · Party ·          │
                │  Stock · Ledger/Voucher · Order        │
                └────────────────────────────────────────┘

                               ▲ ▼

     every one of the 98 apps reads and writes these, and only these
```

One stock number, not one per channel. The last piece sold at the Surat counter disappears from Myntra and Flipkart in the same instant — not three hours later as a cancellation, because cancellations are what a seller rating is lost to.

Accepted — not ordered — is what counts. You order 100 metres. 100 arrive. Quality accepts 96. Most systems increase stock by 100 and claim tax credit on 100. This one increases stock by **96**, claims input credit on **96**, raises a debit note for the 4 rejected, and lowers that mill's accept rate — automatically.

Nothing derived is ever stored. Outstanding, ageing, risk, promise dates, cost per piece and profit per design are recomputed on read. A stored total is a number that can drift away from the documents underneath it; a computed one cannot.

How one garment moves through it

The test of whether this is one system or 98 programs sharing a login: sell a single garment and follow it.

```
sold on a marketplace
        │
        ▼
┌──────────────┐   order lands in one queue, sorted by the time LEFT
│ 15 OMS       │   on its cut-off — not the time it arrived
└───────┬──────┘
        ▼
┌──────────────┐   stock down by one, on EVERY channel, same instant
│ 03 Inventory │
└───────┬──────┘
        ▼
┌──────────────┐   picked from the named bin, in walking order, filmed
│ 10 Warehouse │
└───────┬──────┘
        ▼
┌──────────────┐   cheapest and fastest both known before booking;
│ 11 Logistics │   COD collected at the door reconciled to the bank
└───────┬──────┘
        ▼
┌──────────────┐   revenue and GST posted through ONE posting engine
│ 12 Accounting│   — entries balance or they do not post
└───────┬──────┘
        ▼
┌──────────────┐   weeks later the payout is matched to the paise, and
│ 14 Settlement│   any shortfall is named and claimed before it expires
└───────┬──────┘
        ▼
┌──────────────┐   the karigar who stitched it was paid for it, per raw
│ 08 + 16 Make │   piece, whether or not the piece completed a set
│    and pay   │
└───────┬──────┘
        ▼
┌──────────────┐   every step a live figure — and every figure clicks
│ 21 Dashboard │   down to the record that produced it
└──────────────┘

   one transaction · eight modules · one database
```

Every module and every app

Listed in build order. Each app is marked **working today** or **designed, not yet built** — nothing is described as finished that is not.

Module 01 · Platform

The spine the whole house runs on.

Not a module you open — the layer underneath all 21. Who can see what, how Vastrangam is configured, and a record of everything that ever happened.

Reads from: Every module **Writes to:** Every module

App	What it does	State
Identity, Settings & Audit	Users, roles and permissions, company switching, tax and numbering setup — and an immutable record of everything that ever happened.	designed, not yet built
Ask & Print	At an exhibition in Hyderabad, send one line from your phone: “ledger Kalamandir”, “print slips”. It comes back as a PDF, or it prints at the Surat office — with nothing plugged into your phone and nothing at the office open to the internet.	working today
Communications	WhatsApp commands and broadcasts, email and SMS, and the handful of scheduled jobs that carry a nudge to the right person without anyone remembering to send it. Every capability here has more than one interchangeable provider — none of them is ever the source of a figure the business reports.	designed, not yet built
Data Privacy & Consent	What a person’s data may be used for, captured as consent at the point it is given and honoured everywhere downstream — including the right to have it corrected or removed. Retention (how long a record is kept) and deletion (a person’s right to have their own data removed) are two different policies, tracked separately, because a rule that keeps records for the law and a request from a person to be forgotten do not resolve the same way.	designed, not yet built
Payment Data Scope	A written statement of exactly which systems ever see a card or bank credential, and which never do — because every card-capable screen in this system hands that moment to a payment provider’s own secured field and never stores or even passes the number through application code. The statement is what an auditor or a partner asks for before they will connect to this system.	designed, not yet built

Module 02 · Design & Sampling

A style exists on paper before it exists as stock.

A product moves from a first idea to something the business can actually make and sell — specification, sample rounds, costed trials and sign-off — before it is ever entered as a catalog record. Building this first means Inventory & Catalog never has to invent a style it has no real specification for.

Reads from: CRM **Writes to:** Inventory & Catalog · Manufacturing

App	What it does	State
PLM & Development	Concept to a design that can actually be made: fabric and trim specification, sample rounds with the mill, costed trials against a target price, and sign-off — every version kept, so last season's costing is still there.	designed, not yet built
Design / IP Register	What protects a design once it exists — trademark or copyright status, the date it was first shown, and a flag the moment a near-identical listing turns up elsewhere. Every version already kept by PLM & Development gets a filed status here instead of just a date stamp, because a design with no ownership record on file is a design nobody can defend.	designed, not yet built

Module 03 · Inventory & Catalog

One stock number everyone trusts.

The most important number in the house: one quantity per design and size, per godown, per stage — greige, dyed, in stitching, finished, listed. Read and written by every other module. And one product record every marketplace lists from.

Reads from: Design & Sampling · Every module **Writes to:** Every module

App	What it does	State
Stock	Live quantity by design, size and location, fabric in metres and pieces in numbers, with reorder alerts, lot tracking, set kits and dead-stock ageing.	designed, not yet built
Catalog / PIM	One record per design — fabric, work, length, colour, size chart, images, HSN, MRP and what each panel actually sells it at — scored for Myntra and Amazon readiness before it lists. It also carries the two things everything downstream needs: the code each panel knows the design by, mapped to yours, and the packed size and weight that decide the courier rate and settle every weight dispute. Every listing's state on every panel is here too — live, waiting for approval, blocked, archived — with the quality score that decides whether anyone sees it.	designed, not yet built
Kit & Combo SKU	A sellable SKU made of component SKUs — a three-piece set sold as one listing. Selling the kit decrements each component at order time, so stock is right for every piece in the set, not just the set itself.	designed, not yet built
Master-Data Hygiene	Duplicate detection and merge across customers, vendors and designs, and a dead-stock register for what has not moved in months. Protects every downstream report from the same record existing twice under two names.	designed, not yet built

Module 04 · CRM

Know every boutique, chain and customer completely.

One record per party — a Kalamandir or a Rajmandir, a Surat walk-in or a Myntra buyer — carrying every enquiry, order, return, agreement and conversation, whichever channel it arrived on.

Reads from: Every module **Writes to:** Sales · E-commerce / OMS · Marketing

App	What it does	State
CRM & Customer 360	Enquiry to confirmed order, then the full lifetime: what they bought, what came back, what they are worth and which new range to show them first.	working today
Documents & eSign	Mill agreements, job-work contracts, signed delivery challans, export documents and boutique credit terms filed against the party or order they belong to — found by that record, not by hunting through a folder.	working today
Helpdesk & Live Chat	A boutique asking where its parcel is, or a customer asking about a size — the question becomes a ticket tied to the order, with the whole history already open.	working today
Forms & Feedback (NPS)	A short form after delivery, and the score it produces attached to the design or item it is actually about — not just the buyer — so a complaint-prone item surfaces as a pattern instead of a scatter of individual gripes.	designed, not yet built

Module 05 · Sales

Counter, wholesale, website and export — one order book.

The Surat counter, the boutique wholesale book, the website and the export shipment all write to the same order and draw on the same stock number. And the parcel is followed to the door, because a sale is not done until the COD money is in.

Reads from: Inventory & Catalog · CRM · Warehouse · Logistics **Writes to:** Inventory & Catalog · Accounting & GST · Warehouse · Logistics

App	What it does	State
D2C Sales	Orders from your own storefront, cart to dispatch, with loyalty and partial COD on a ₹4,400 anarkali.	working today
B2B & Credit	Boutique and chain orders on credit limits and tier pricing, with outstanding aged against each party's own agreed terms.	working today
Export	Commercial invoice, packing list, LUT bond and IGST-refund tracking for the Gulf and UK buyers.	working today
POS	Counter billing at Udhna that draws on the same stock as the website — no second stock register.	working today
Quotes & Proforma	Send a quote, convert it to a confirmed order in one click.	working today
Couriers & AWB	Book the parcel on the order, compare couriers for that pin code, print the label with the design code on it, and follow the AWB to the door.	designed, not yet built
Subscriptions	A schedule that raises its own invoice on its cycle and follows up on its own when a payment fails — for anything sold as a standing order rather than a one-off.	designed, not yet built

Module 06 · Planning & Requirements (MRP)

Turn what is selling into what to buy and make.

Confirmed orders and demand history have to become a plan before Purchase can buy anything or Manufacturing can start anything — otherwise buying and making are both just guessing. This module sits between the two: it reads what is actually selling and what is already committed, and turns that into requirement, not the other way around.

Reads from: Sales · E-commerce / OMS · Inventory & Catalog **Writes to:** Purchase · Manufacturing

App	What it does	State
Demand Forecast & Signal	What sold, by SKU and by period, turned into a short-term forecast — the number every requisition and every production order downstream is measured against instead of a guess.	designed, not yet built
Requirement Explosion (MRP run)	Confirmed demand exploded through the bill of materials into what raw material to buy and what to produce, and by when — so a purchase requisition or a production order always traces back to real demand, never to a feeling that stock is getting low.	designed, not yet built
Open-to-Buy / Budget Ceiling	A spending ceiling set per period regardless of what the demand signal suggests, so a real signal cannot on its own commit more money than the business has decided to risk this season. Every requisition the MRP run drafts is checked against what is left of the ceiling before it goes to Purchase.	designed, not yet built

Module 07 · Purchase

Nothing over-billed by a mill gets paid.

The buy side end to end — mills, dyers, job workers and packing suppliers — with the control that stops you paying for metres you rejected.

Reads from: Inventory & Catalog · Planning & Requirements (MRP) · Manufacturing **Writes to:** Inventory & Catalog · Accounting & GST · Quality & Compliance

App	What it does	State
Procurement	Enquiry to purchase order to goods receipt, with a strict three-way match: you ordered 100 metres, 100 arrived, quality accepted 96, and the bill is only cleared for 96.	working today
Vendor Management	Mill 360 — payables, ageing, a real risk score from accept rate and spend concentration, and sourcing that follows performance rather than habit.	working today
Insurance Register	What cover exists over stock in transit, stock in the warehouse and product liability, matched against what is actually moving through Purchase and Warehouse right now — so real value in transit is never quietly running with no cover tracked anywhere.	designed, not yet built

Module 08 · Manufacturing

Know what a piece really costs to make.

From the cut plan to the finished piece — what each karigar earned, what the dyer charged, what the zari cost, and what that design actually cost before you priced it.

Reads from: Purchase · Planning & Requirements (MRP) · Design & Sampling **Writes to:** Inventory & Catalog · HR & Payroll · Accounting & GST · Quality & Compliance

App	What it does	State
Production Orders	Cutting, stitching, embroidery, washing, finishing and checking — your own stages, with work-in-progress visible at each and nothing lost at the dyer.	designed, not yet built
Piece-rate & Contractors	Karigars paid by the piece: pooled set completion, per-garment rates, alterations, rework and advances resolved into one payout.	designed, not yet built
BOM & Consumption	What each design consumes — metres of fabric, zari, lining, buttons, packing — costed at today's mill rates.	designed, not yet built
Maintenance	Machines and the building: what is due for service, when it was last done, what it cost, and what stopped while it was down.	designed, not yet built

Module 09 · Quality & Compliance

Certify what was received and what was made.

Quality inspection used to live buried as one step inside Manufacturing; it stands on its own here because a rejection at goods-receipt and a rejection on the production floor are the same discipline, and because the certificates a business holds — the proof it follows a standard — belong next to the inspections that back them up, not scattered across email.

Reads from: Purchase · Manufacturing **Writes to:** Purchase · Manufacturing · Inventory & Catalog

App	What it does	State
Quality Control	Accept, reject or send for rework, with reasons that feed the mill's accept rate and the karigar's record.	designed, not yet built
Certificate & Compliance Register	Every standard the business or a vendor holds — a safety, labour or environmental certification — with its issue date, its expiry, and the audit that backs it, so a certificate about to lapse is visible before a buyer asks for it and finds it already expired. What this register tracks is what a sustainability report downstream is actually built from.	designed, not yet built

Module 10 · Warehouse

Pick the right design first time — and prove what you sent.

Bin-level instructions and barcode scanning so the right piece leaves the godown and stock stays honest — and a recording of each parcel being packed, because a wrong-return claim is settled by footage, not by argument.

Reads from: Sales · E-commerce / OMS · Inventory & Catalog **Writes to:** Inventory & Catalog · Sales · E-commerce / OMS

App	What it does	State
Picking & Bins	Pick lists in walking order through the godown, by design and size, so nobody crosses the floor twice.	designed, not yet built
Barcode Operations	Scan to pick, pack, dispatch and run a physical stock count from a phone — the same scan whether the order came from a marketplace, your Shopify site or the counter.	designed, not yet built
Packing Video	Every parcel filmed as it is packed and indexed by its order number, so when a panel says the wrong piece was sent, the clip goes into the claim.	designed, not yet built

Module 11 · Logistics

The courier network — rates, failed deliveries and the COD money.

Booking one parcel happens on the order. This module is the network behind it: what Delhivery, Blue Dart and the rest charge to that pin code before you pick one, what happens to a delivery that fails in a small town, and whether the cash collected at the door reached your bank.

Reads from: Sales · E-commerce / OMS · Warehouse **Writes to:** Accounting & GST · Sales · E-commerce / OMS

App	What it does	State
Rates & Zones	Every courier's rate card by zone, weight slab and service — so the cheapest and the fastest option for this parcel are both known before it is booked.	designed, not yet built
NDR & RTO Rescue	A failed delivery worked while it can still be saved — reattempt, call, correct the address — before it becomes a return you pay for twice.	designed, not yet built
COD Remittance	What the courier collected at the door against what reached the Surat account, parcel by parcel, with every shortfall named and aged.	designed, not yet built
Handover & Manifest	What is expected out today against what the pickup boy actually took, per courier and per service. The manifest to hand him, the one-time code to confirm it, and a signed note of what was left behind — so a parcel lost between the packing table and the van has an owner.	designed, not yet built
Fleet	Your own delivery vehicles, if you run any — what each trip cost, what is due for service, and what that adds to freight. Optional; most businesses run couriers only.	designed, not yet built

Module 12 · Accounting & GST

Books that always balance — and no BUSY needed.

A full double-entry ledger built for Indian compliance, keeping the books itself. B2B sales, returns, mill purchases, payments and receipts are entered by hand because a person decides them; every website, marketplace and counter sale posts itself.

Reads from: Every module **Writes to:** Finance Reports · Treasury & Financial Planning

App	What it does	State
Accounting	Double-entry books where every voucher balances and the trial balance always ties.	designed, not yet built
Invoicing	GST tax invoices and receipts, worked out from the lines to the paise. Where a panel raises its own invoice you keep both numbers on the order — theirs and your own series — so the panel's paperwork and your books point at the same sale.	designed, not yet built
Expenses	Spend captured by category with approvals, and bill OCR to save typing.	designed, not yet built
GST & Tax	CGST, SGST, IGST, TDS, TCS, input credit, GSTR-1 and GSTR-3B — filed per registration, so a group where one company is registered and another is not files exactly what each one owes and nothing it does not.	designed, not yet built
ITC Reconciliation	Every input credit matched against the government's own GSTR-2A/2B before a return is filed, so a credit claimed is a credit that is actually on record.	designed, not yet built
Receivables, Payables & PDC	Payments and receipts allocated against named open invoices, and a register for post-dated cheques that posts only on the date they are realised, not the date they were written.	designed, not yet built
Fixed Assets & Depreciation	The asset register with both Straight-Line and Written-Down-Value depreciation tracked side by side, and disposal posting its gain or loss straight to the books.	designed, not yet built
Year-End Close & Period Lock	Profit-and-loss accounts reset and balance-sheet accounts carry forward at year end, and a reviewed period locks against backdated edits until an admin unlocks it — an act that is itself logged.	designed, not yet built
Finance Reports	P&L, balance sheet, and profit by channel, design and SKU — so you know which anarkali actually earned money after commission, shipping and returns.	designed, not yet built

Module 13 · Treasury & Financial Planning

Know what cash is coming, not just what already arrived.

Accounting records what happened; this module is concerned with what happens next — how much cash is actually expected, when, and whether spend against a budget is on track before the month closes and turns the answer into history.

Reads from: Accounting & GST · Sales · Purchase **Writes to:** Accounting & GST

App	What it does	State
Cash Flow Forecast	Expected receipts and expected payments laid out by week, drawn from open invoices and open bills rather than typed in by hand — so a cash shortfall is visible weeks before the date it would actually bite.	designed, not yet built
Banking & Reconciliation	Bank statement lines matched against the ledger's own record of what should have moved, with anything unmatched surfaced instead of silently carried forward.	designed, not yet built
Budget vs Actual	A budget set per category and period, and actual spend from Accounting tracked against it as the period runs — not only after it closes, when the only thing left to do is explain the variance.	designed, not yet built

Module 14 · Settlement

Get paid what the panels owe you — cycle by cycle.

Matching one payout to one order line happens in OMS. This is the level above: the settlement cycle each panel runs, the commission it actually charged against the rate card it published, and the TCS it deducted in your name.

Reads from: E-commerce / OMS · Accounting & GST **Writes to:** Accounting & GST

App	What it does	State
Payout Cycles	Every settlement cycle each panel runs — what it should pay, what actually landed in the bank, and on which day — so a late payout is visible the day it is late, not at month end.	designed, not yet built
Fee & Commission Audit	The commission a panel publishes for a category against what it actually took, style by style. A quiet rate change is caught the first time it is applied, not at year end — and your seller tier sits on the same screen, because the tier is what the rate card hangs off, and slipping out of one quietly costs more than any single deduction.	designed, not yet built
TCS & TDS Register	Every rupee the panels deducted as TCS, and TDS on job work, matched against the portal's own figures — so the credit you claim is the credit you are owed.	designed, not yet built

Module 15 · E-commerce / OMS

Seven panels, one queue — and every rupee accounted for.

Stop logging into Myntra, then Flipkart, then Ajio. Every marketplace order lands in one pipeline and your stock goes out to all of them — then the money side closes out in the same module: what the panel paid, what it kept as commission, what came back, and what it still owes you.

Reads from: Inventory & Catalog · CRM · Sales · Accounting & GST · Logistics · Settlement **Writes to:** Inventory & Catalog · Accounting & GST · Warehouse · Logistics · Settlement

App	What it does	State
Marketplace OMS	Myntra, Flipkart, Ajio, Amazon, Meesho, Nykaa and JioMart in a single queue — processed all together, channel-wise, or design-wise. The stages the panels really use, with the right cut-off counting down on each order — a quick-commerce or air-shipped order is not due at the same hour as a standard one. Priority orders at the top, and the day grouped by design so a Muskan Purple is picked once for eleven parcels instead of eleven times.	working today
Order Management	One pipeline from new to delivered, whether the order came from a seller panel, your Shopify or WooCommerce site, a dealer or the counter.	working today
Manual Data Check	The order and return sheets you already download from the panels, and the offline registers from the three shops — one file or a whole ZIP — read back as ten cross-checks: net sale after commission and fees, month, design, state, wrong returns, SPF claims, ads, payouts and GST. Every figure clicks through to the transactions behind it.	designed, not yet built
Reconciliation	Match every marketplace payout to the order line that earned it, and expose the gap.	designed, not yet built
Claims & Disputes	Weight disputes, SPF shortfalls, parcels lost in transit and returns that came back with a different piece inside — filed as claims with the packing footage attached, and answered before they close. A claim awaiting your reply is money; one closed for no response is nothing, so the days left sit beside the amount.	designed, not yet built
Returns / RMA	Customer returns, courier returns and wrong returns kept apart — because only one of the three is really your fault, and only one of them turns into dead stock.	designed, not yet built
Channels & Storefronts	Connect a channel once and it stays in step: catalogue out, price out, stock out, orders in. Seven marketplaces and any website platform — Shopify, WooCommerce, Magento, BigCommerce, Wix or a custom site over its own API — each switchable without touching your data. Shopsy and any other storefront a channel runs alongside its main one counts as its own channel here. Where a channel has no open interface, its own downloaded report is a first-class way in. A channel may also know you by a different trading name — that is a label on the channel, not a second company, so it tags the order and the payout without ever splitting your books.	designed, not yet built
Labels & Documents	The panel gives you a PDF; this hands the packing table something it can work from. Cropped to 4×6 for every channel, your design code printed large where the panel left it off, the invoice and slip merged behind it, and the whole batch to the label printer in one job. Reprint one parcel without redoing the lot — and no customer's name and address is ever uploaded to an outside website to be cropped.	designed, not yet built
Listing & Catalog Manager	Bulk-create and bulk-edit listings across every channel from the one product record in Inventory & Catalog, and catch the mismatches that quietly cost sales: listed but out of stock, or in stock but never listed.	designed, not yet built
Size / Fit Recommendation AI	A fit suggestion at the point of purchase, built from the item's own measurements and the return history of buyers who picked each size — aimed straight at the return reason that costs the most: the right item in the wrong size.	designed, not yet built

App	What it does	State
AR / Virtual Try-On	A way to see drape, fit and colour on a screen before buying, for items where a flat photo alone leaves too much to guess.	designed, not yet built

Module 16 · HR & Payroll

Pay everyone right, on time.

Office staff on a monthly salary and karigars paid by the piece, in one register, with attendance driving both and the festival advance already deducted.

Reads from: Manufacturing **Writes to:** Accounting & GST

App	What it does	State
Staff & Contractors	Attendance marked by tap, effective-dated salary, and karigar piece-rate earnings in a single register.	designed, not yet built
Time-off & Advances	Leave, Diwali advances, and exactly how they change this month's payout before you approve it.	designed, not yet built
Appraisal & Hiring	Performance reviews and a hiring pipeline that ends in an employee record.	designed, not yet built
Payout Execution	Where the calculation in the earnings register actually turns into money leaving the business — bank batch, UPI, cash against a signed receipt — with the method and the reference recorded against every payout, so the register's total and the money that actually moved can always be checked against each other.	designed, not yet built

Module 17 · Marketing

Sell more without cutting the price.

Plan the festive calendar, run the campaigns, and let rules keep you competitive on the panels without giving the margin away.

Reads from: Inventory & Catalog · CRM **Writes to:** Sales · E-commerce / OMS

App	What it does	State
Social Calendar	Plan and publish across every channel from one calendar.	designed, not yet built
Campaigns	Email, SMS and WhatsApp campaigns measured on real revenue, not opens.	designed, not yet built
Repricing Engine	Rules per panel and per design — floor, ceiling, match-lowest and a festive override — so a Diwali sale does not quietly go below cost. And what each change actually did: a design whose orders fell after a price rise shows as exactly that, next to the rule that raised it.	designed, not yet built
Automation	If this happens, do that — across any module, without writing code.	designed, not yet built
Blog & Pages	How to drape it, what to wear it to, which fabric for which season — written, scheduled and published to your own site with the meta and internal links already set.	designed, not yet built
Events	Trade shows and exhibitions worked as a channel of their own — booth, budget and every lead captured on the floor landing straight in CRM instead of on a stack of business cards.	designed, not yet built
Markdown / Clearance Optimization	The same rule engine that reprices for competitiveness, aimed at ageing stock instead: when to start discounting it and by how much, before it becomes a warehouse write-off rather than a sale at a lower margin.	designed, not yet built

Module 18 · AI Content Engine

Write it, shoot it, cut it — from your own catalogue.

Listings, ads, reels and product photography generated from your own designs, in a voice that sounds like one person from Surat rather than a template — so the words match the piece and the picture is the size Myntra actually wants.

Reads from: Inventory & Catalog **Writes to:** Marketing · E-commerce / OMS

App	What it does	State
Content Engine	Fourteen stages in your own voice — buyer psychology, competitor reading, hooks, the product description, marketplace copy for Amazon and Myntra, ad variations, reel scripts, song lyrics for the reel, the calendar, size chart and alt text.	designed, not yet built
Image Studio	A phone photo becomes a listing image: layers, free transform, background removal, Myntra 1080×1440 and every other channel preset, watermark and SEO alt text.	designed, not yet built
Video Studio	Text and image to video, reels and ad cuts sized for every channel.	designed, not yet built
Design Studio	Banners, festive creatives and thumbnails — templates, layers, undo and redo, any colour, exact sizing and stock elements, exporting PNG, JPG or PDF at whatever size the panel or the printer asks for.	designed, not yet built
Publisher	One push sends the listing, images and copy to the website and every panel, and reports back what went live and what a panel rejected, with the reason.	designed, not yet built

Module 19 · SEO, AEO & AIO

Be found by a search box, an answer box and an AI.

Content already exists once this module is reached; here it is made findable — by a traditional search engine, by the answer box above the results, and by the AI assistants now answering shopping questions directly instead of sending someone to a results page.

Reads from: Inventory & Catalog · AI Content Engine **Writes to:** Marketing

App	What it does	State
Technical SEO & Schema	Structured data, sitemaps and page-level technical checks against your own storefront and content pages, so a search engine can read what a page is actually about instead of guessing from the text alone.	designed, not yet built
Answer-Engine Optimization	Content shaped to be quoted directly by an answer box or a voice assistant — a clear, citable answer near the top of the page — rather than written only for a person to scroll through.	designed, not yet built
AI-Engine Visibility Tracking	Whether and how this business is actually cited when someone asks an AI assistant a shopping question in this category, tracked over time — the same discipline as rank tracking, aimed at a newer kind of result page.	designed, not yet built

Module 20 · Projects & Collaboration

The work that is not an order — and the talking around it.

An exhibition in Hyderabad, a boutique’s custom order, a new godown fit-out, a legal matter with a supplier. Work that is not a sales order still has a deadline, a cost and documents — and it belongs on the same records as everything else.

Reads from: CRM · Sales · HR & Payroll · Inventory & Catalog **Writes to:** Accounting & GST · HR & Payroll · CRM

App	What it does	State
Projects & Cases	An exhibition, a custom order for a chain, a fit-out or a dispute — stages you define, owners, deadlines, documents, hours and real cost, all on one record the ledger can see.	designed, not yet built
Timesheets & Planning	Who is on what this week and the hours that actually went in — against a project, an exhibition or a machine — with billable and non-billable kept apart.	designed, not yet built
Approvals	One queue for everything waiting on a yes: a mill purchase order, a boutique discount, a leave day, a credit note, a payment. The rule that sent it there is next to it, and the decision goes on the record.	designed, not yet built
Forum	Questions and answers that outlive a chat — for customers, dealers or staff — with the useful ones kept where the next person will actually find them.	designed, not yet built
Discuss	The conversation attached to the record it is about — this order, this mill bill, this dispute — so a year later the reason for the decision is still sitting beside it.	designed, not yet built
Knowledge Base	A searchable internal wiki of standard operating procedures, scoped to the role it applies to, so how a task is meant to be done is written down once instead of carried in one person’s head.	designed, not yet built

Module 21 · Dashboard & BI

See the whole house without asking anyone.

Every number rolls up here as work happens — the day’s marketplace orders, what the karigars finished, what is still lying at the dyer, what the mills are owed. No exports, no waiting for month-end.

Reads from: Every module **Writes to:** —

App	What it does	State
CEO Dashboard	Cash, sales by channel, stock by design, profit per piece and the alerts that matter — one screen, refreshed as the day runs.	working today
Report Builder	Drag the fields you want into a report and save it for the whole team.	working today
Group Consolidation	Ethnic Fashion, Vastrangam and Adini Couture as one set of figures, inter-company transfers removed, so the group position is real rather than three spreadsheets added together. Adini Couture has no registration of its own and mainly does job work — it still counts in the group, without being pulled into a return it does not belong in. Add the fourth company the day you open it.	working today
Excel Dashboard Builder	A full workbook — financial summary, HR, purchase, sales, inventory and production, GST, expenses — generated from the live records behind every other screen, with each company shown as its own row and a consolidated row that is a formula over them, never a separately typed total.	designed, not yet built
ESG / Sustainability Reporting	Water usage, chemical compliance, waste and packaging, reported from the same certificate and audit records Quality & Compliance already keeps — so a sustainability report is a query over evidence already on file, not a separate exercise assembled once a year from scratch.	designed, not yet built

Companies and channels — how many is up to you

You have three companies and sell on seven marketplaces. Neither of those is a setting this system was built around, and neither is a ceiling. A company is a **row**. A channel is a **row**. Every business record carries the company it belongs to, and every sale carries the channel it came through. Ten companies selling on ten channels each is the same three tables and the same code as three and seven.

```
COMPANIES (a row each)          CHANNELS (a row each, per company)

┌──────────────┐                ┌────────────────────────────────────────┐
│ Vastrangam   │───────────────▶│ D2C · Amazon · Myntra · Flipkart ·     │
│ Ethnic (GF)  │───────────────▶│ Ajio · Meesho · Nykaa · POS ·          │
│ Adini        │───────────────▶│ B2B desk · Export buyer · …            │
│ …the eighth  │───────────────▶│ …the eleventh                          │
└──────────────┘                └────────────────────────────────────────┘

       │                                          │
       └──────────────┬───────────────────────────┘
                      ▼
           ONE stock number per SKU
     the channel is on the sale, never on the stock
```

Three things this buys you, and one it deliberately refuses.

Each company's books are its own. Its trial balance balances on its own. No report can reach across into another company's rows — not by convention, but because a journal line can only point at an account that belongs to the same company, and a test checks that no line anywhere ever does.

The group is the sum minus what you sold yourselves. When Vastrangam sells to Ethnic, that is revenue in one set of books and cost in another. Adding the companies up would report a group turnover the group never earned from the outside world. Every entry that names a sister company is eliminated at group level, and the consolidation returns all three numbers — gross, eliminated, group — so you can see the elimination rather than take it on trust.

The channel is a dimension of the sale, never of the stock. You can read this month by channel, by company, or by both. What you cannot do is keep a separate stock number per channel, and that is on purpose: the last piece sold on one marketplace has to vanish from the other ten at that instant, which per-channel inventory cannot do.

And the tool follows your sheets. Drop a workbook into the Data Studio and the report's columns come from the sheets that are actually in it. Two companies today gives two pairs of columns; a fourth company is a new sheet in the workbook, not a new version of the software.

> **This is checked, not claimed.** `core/tests/core.test.js` *builds ten companies with ten channels each — a hundred channels — posts an order down every one plus ten inter-company sales, and asserts every company's books balance, that no journal line points at another company's account, and that the group figure is the plain sum* **minus** *inter-company trade: ₹2,10,500 gross, ₹50,000 eliminated, ₹1,60,500 group. It then calls the same builder for eleven companies and eleven channels with no code changed. The Data Studio's own tests do the matching thing on the reporting side: ten companies in one workbook produce ten pairs of columns.*

The rules that hold everywhere

1. **No app depends on any single outside company.** Every capability — books, marketplaces, AI writing, couriers, payments, messaging, storage, GST, printing, barcode — has several interchangeable providers and a by-hand option, so the system works with nothing connected at all. **A provider named as the source of a figure is a bug**; providers move messages and money, the ledger originates numbers.
2. **The books are this system's own.** No other accounting package is required, ever. Tally, BUSY and Zoho remain available for anyone already running one — nothing assumes them, and no figure is ever sourced from one.
3. **Nothing ever asks for an account password.** Outside services connect with a scoped, revocable key. *This system will never ask you for a marketplace, bank or account password. If any screen ever does, it is not this system.*
4. **Money is integer paise, never a floating-point number**, because float arithmetic accumulates the error that eventually shows up as a trial balance that will not tie.
5. **Values that change over time are effective-dated** — a salary, a price, a tax rate, a commission. March payroll resolves the salary in force *in March*. A missing value is an error, never silently treated as zero.

6. **Nothing is deleted, only deactivated.** A person who leaves keeps their name attached to years of earnings, approvals and audit rows that must still resolve.
7. **The audit trail has no off switch.** Eight years, before-and-after values, as the MCA rule requires — because an audit trail that can be switched off is one that gets silenced exactly when it matters.
8. **Gates, not warnings.** Each app refuses one thing outright, because a warning gets clicked through on a busy afternoon. A cash-on-delivery order cannot be packed below its advance. A bill for more than was accepted cannot be paid. Every gate is also a self-test.
9. **It is not trained. It is built.** No model learns from your data. Every rule is written down, visible on the Wiring screen, and checked by a self-test — so it is right on day one.
10. **You can reach it from anywhere, but it cannot be reached into.** Ask & Print takes a plain line from your phone and sends a PDF back. The office reaches out; the internet never reaches in.

How it is verified

Nothing ships because it looked right on a screen.

1. **The arithmetic, with no screen involved.** Each engine runs in isolation and its self-tests execute against seeded data.
2. **Every screen and every control, in a real browser.** Each build opens in headless Chromium; every screen is visited and every interactive control on it is clicked. Any console error fails the build.
3. **The real job, with the result asserted.** Not "does the button click" but "did the thing happen". A control that looks alive but changes nothing fails the build.
4. **Against the owner's own figures.** Where the business already knows the answer, the software has to reproduce it exactly — the karigar costing run must return **25,307 sets, 59,110 pieces and ₹26,90,062** across 143 designs and 29 karigar units, with the 5 no-rate designs flagged rather than guessed. A mismatch is a bug, not a rounding difference.
5. **A structural audit.** Every "comes from" on every Wiring screen must name a module that actually exists, no vendor name may ever be the source of a figure, and the app count in every file must match this one.
6. **The shipped copy, not the working copy.** The packaged archive is extracted and re-tested in the folder a customer would open it in, because a packaging step that quietly renames a file breaks nothing until it is in somebody's Downloads folder.

The honesty charter

This is the standard the build is held to, and it is written down because a standard nobody wrote down is a standard nobody can be held to.

1. **Nothing is called finished that is not.** Every deliverable is labelled tool, stub, mockup or spec. A mockup stays labelled a mockup even when a working version would be more impressive to show. Generation features stay badged as mockups until a real paid API is actually wired.
2. **Counts are counted, never claimed.** Every module and app figure on this page is read from the canonical module list when the page is built. No number here was typed by hand.

3. **Progress is reported as it is.** If tests fail, the failure is shown with its output. If a step was skipped, it is named as skipped. "Done" means implemented, tested and checked against the original request — not "the code has been written".
4. **The gap is stated, not buried.** 16 of 98 apps work today. The other 82 are designed and specified. Those 16 still run on their own storage, and rewiring them onto the shared core is the first job of Module 01 — until that is done they are good tools, not yet one system.
5. **Uncertainty is surfaced, not smoothed over.** Where something cannot be verified, it is reported as unverified rather than presented as fact.

Vastrangam BOS · one business, one brain · 21 modules · 98 apps · one shared data core · 16 working today

---

PART TWO — THE PLAN OF ACTION

One platform. Every function. Wired together.

The build plan for a multi-company operating system covering a Surat ethnic & western fashion manufacturer running three sister companies — D2C, seven marketplaces, B2B and export on one order book, one stock number and one ledger.

21 modules · 98 apps · 16 apps working today · 82 to build. Built in dependency order, Module 01 first through Module 21 last. Every module is finished — every app on the shared database, verified in a browser — before the next begins.

How to read this. Part I is the integration layer: the law the whole system obeys, the data core that makes it one system rather than 21 programs, and the wiring between modules. Part II is the 21 modules one at a time, each with a diagram of how it actually functions. Every section carries a diagram because the point of this document is that you can see the machine working, not just read a list of features.

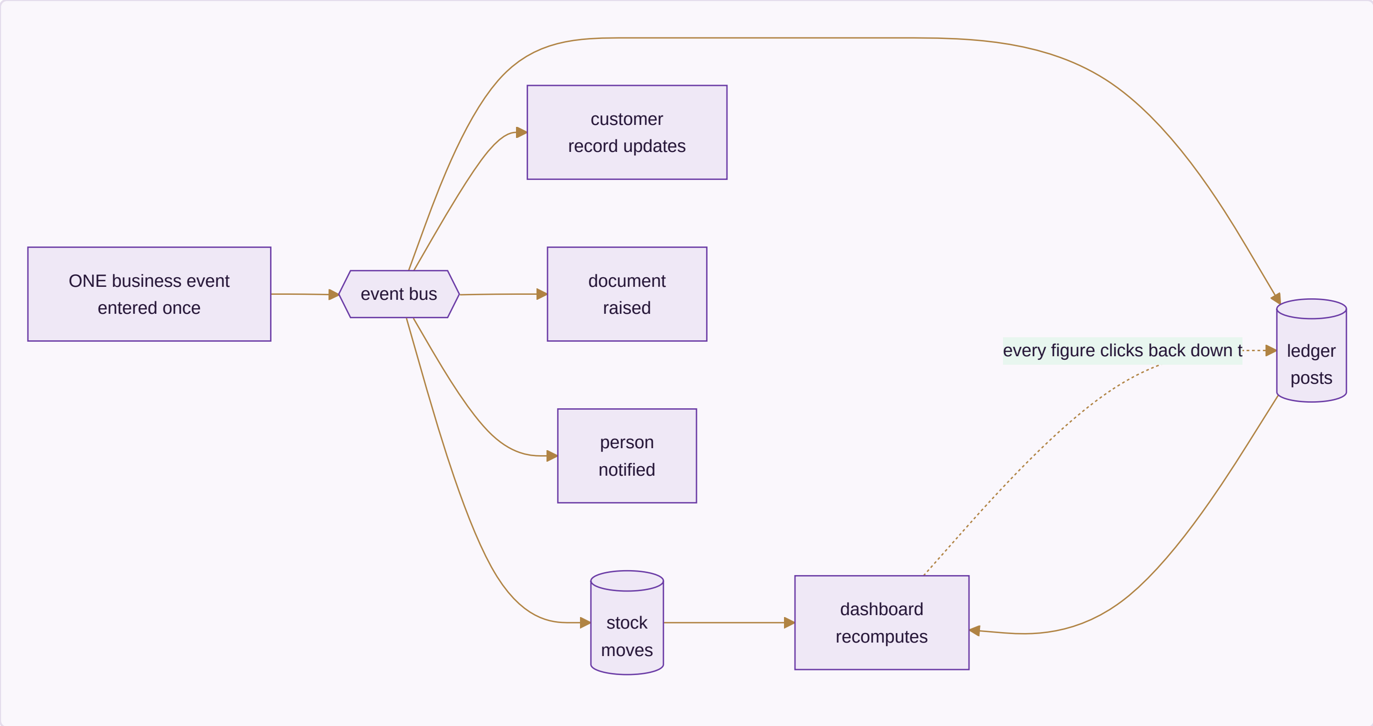
Honest framing. Nothing in this document is described as finished unless it is. Each app is marked **BUILT** (a working file you can open today, carrying its own self-tests) or **SPEC** (designed to this behaviour, not yet written). The count above is the true one: 16 of 98.

PART I — HOW THE SYSTEM WORKS

A0 · THE ONE LAW — INTEGRATION

A business event is entered once and flows everywhere it belongs. No module re-enters data another module already has. No number is maintained separately from the record that created it.

This is the whole design. Everything else in this document serves it.



The test of the law. Any figure on any screen can be clicked down to the record that produced it, and that record down to the document that created it. A dashboard number is a live query over the ledger — never a total someone maintained on the side. If a figure cannot be traced, it is a defect, not a rounding difference.

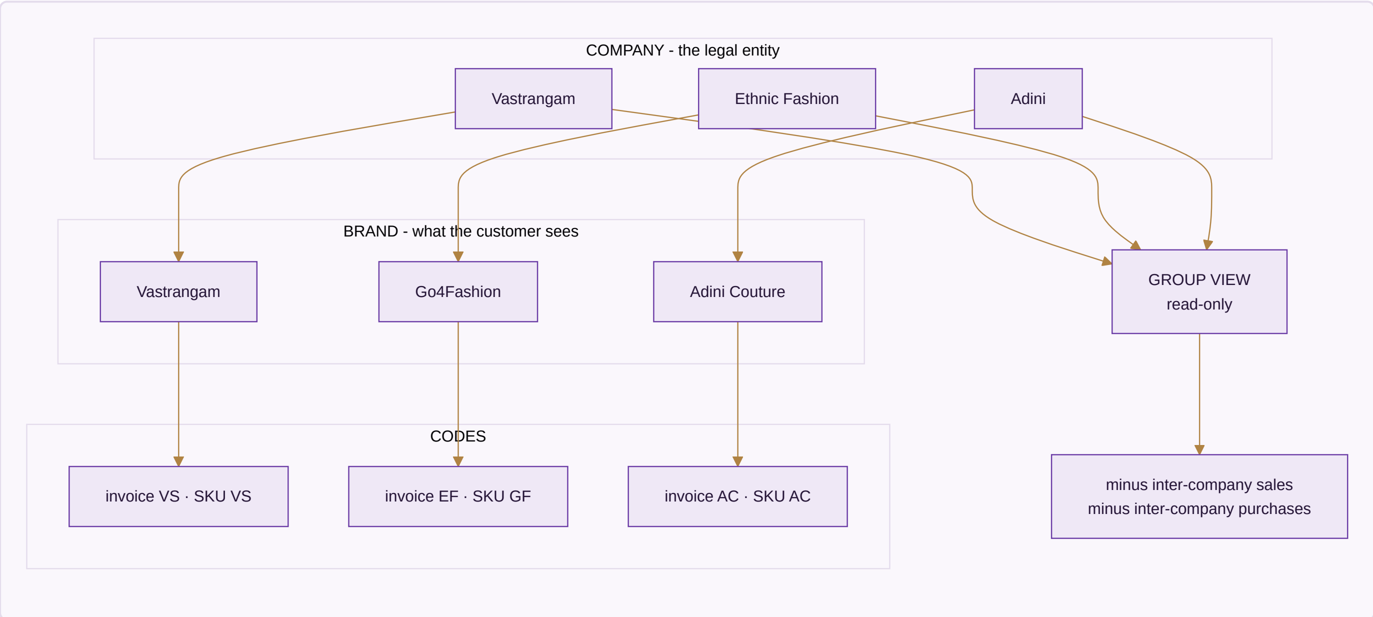
The eight cascades that must fire by themselves

These are the specific chains the platform has to make automatic. If one of them needs a human to re-key something, the system has failed at the only thing that makes it a system.

A single action…	…updates every one of these, in one transaction
A sale, any channel	stock deducted → invoice with GST → ledger (Dr Debtor/Bank, Cr Sales + Output GST) → customer outstanding and lifecycle → settlement expectation → dashboard
A marketplace order pull	sales order created → stock reserved → pick list → fulfilment → on payout, reconciliation and variance check → books
A settlement import	each line matched to its order → commission, fees, TCS, TDS to books → variance → claim raised → bank reconciliation → SKU profit → dashboard
A return	credit note → refund → a wrong return becomes dead stock, never restocked → return cost to P&L → return rate feeds design analytics
A karigar production report	pooled set completion and piece-rate earnings → finished stock in → payout in HR → Karigar Wages posted → cost per piece → design profit
A material purchase	PO → GRN → three-way match → raw stock in → vendor payable → input tax credit → landed cost into COGS
A staff attendance mark	effective-dated salary resolved → payroll → Salaries posted → productivity cost → design cost
A generated image or listing	asset library → product listing on the storefront and each marketplace → marketing calendar → published → campaign return tracked back

A1 · THE THREE COMPANIES

The single most common way a multi-company system goes quietly wrong is collapsing three different ideas into one field. They are kept apart at the root of the database.



Company ≠ brand ≠ prefix. Ethnic Fashion the company trades as Go4Fashion the brand; its invoices read EF and its SKUs read GF. Reports group by *company*. SKUs use the *brand code*. Invoices use the *company prefix*.

Three separate fields, because they genuinely are three different things.

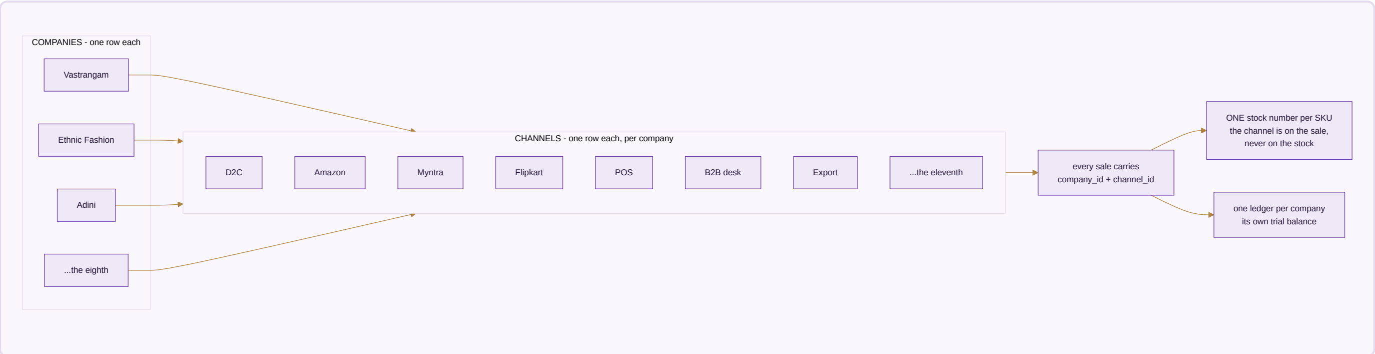
Group is a lens, not an entity. The consolidated view is read-only. Group profit is the sum of the three companies minus inter-company sales and purchases — a real elimination, so moving stock from one pocket to another never inflates group turnover. Editing happens inside a company; the group only reports.

A company without its own registration is still a company. A job-work arm that holds no GSTIN counts in the group figures without being dragged into a return it does not belong in.

Three is today's data, not the design. Nothing above is built for the number three. `companies` is an ordinary table; adding an eighth is inserting a row. The next section is about the other half of that — the channels those companies sell through.

A1b · COMPANIES × CHANNELS — WHY NEITHER NUMBER IS FIXED

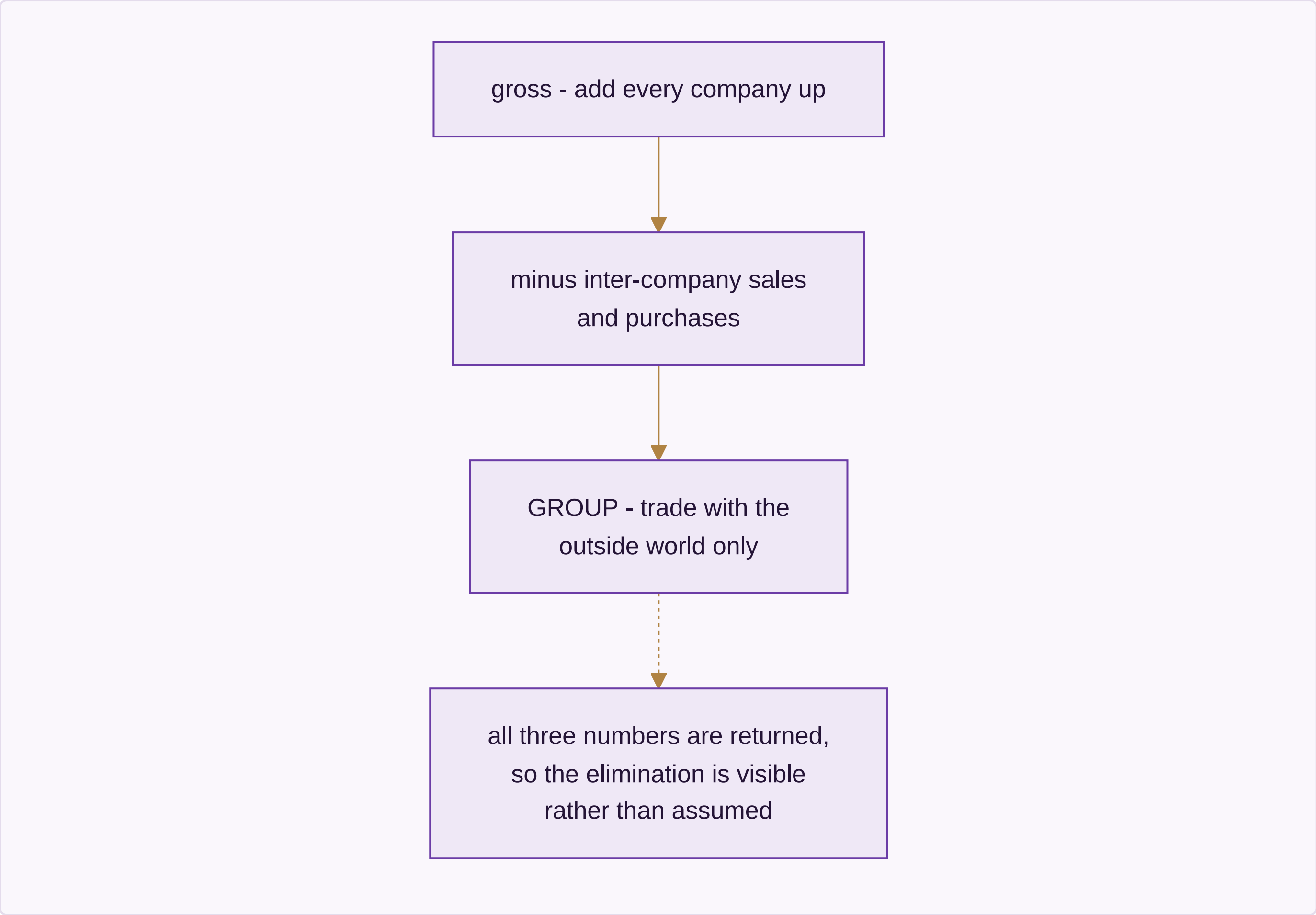
A **channel** is a way a company sells: its own site, a marketplace seller account, a POS counter, a B2B desk, an export buyer. Like a company it is a row, not a column and not a list in the code. Three companies on seven marketplaces is the data this business has now; ten on ten is the same three tables.



Every business row carries its company. A journal line can only point at an account belonging to the same company, so one company cannot read another's figures — checked by a test rather than left to discipline.

The channel is a dimension of the sale. `channel_id` sits on the stock movement and on the journal entry, so any figure can be read by channel, by company, or by both. It deliberately does *not* sit on the stock row: inventory is not per channel, because the last piece sold on one marketplace must vanish from the other ten in the same instant.

The group eliminates what the companies sold each other. An entry whose other side is a sister company carries `counterparty_company_id`, and consolidation removes exactly those.

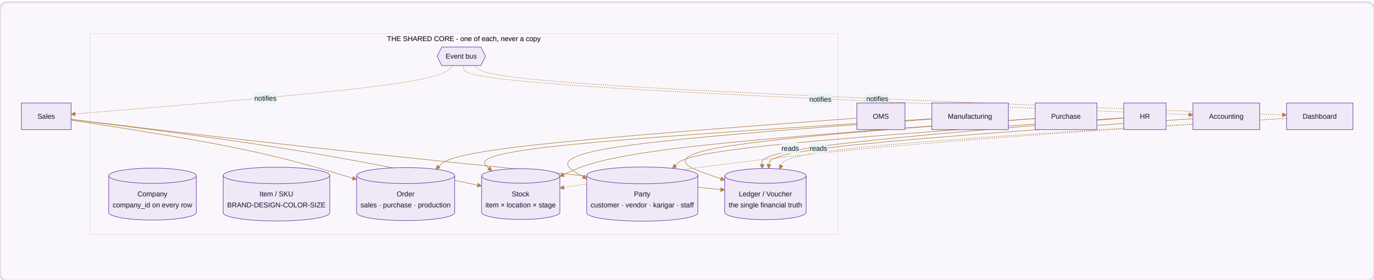


This is proved, not asserted. `core/tests/core.test.js` builds **10 companies × 10 channels = 100 channels**, posts one order down every channel plus ten inter-company sales, and checks each company's figures channel by channel, that every trial balance still balances, that no journal line anywhere points at another company's account, and that the group figure is the sum minus inter-company trade — **₹2,10,500 gross, ₹50,000 eliminated, ₹1,60,500 group**. The same builder is then called for 11 × 11 with nothing in the code changed.

The reporting side matches: the Data Studio detects every `<Company> Sale` / `<Company> Return` sheet pair present in an uploaded workbook and emits one pair of quantity columns per company. A fourth company is a new sheet, not a new release.

A2 · THE UNIFIED DATA CORE

This is the physical reason a sale can touch stock and the books at the same instant: there is one set of master records underneath every module, not one silo per module.



The seven things there is exactly one of. Company · Item/SKU · Party · Stock · Ledger · Order · the event bus that lets modules notify each other. Every business table carries `company_id`, and row-level security enforces it in the database — so even a bug in application code cannot leak one company's data into another's.

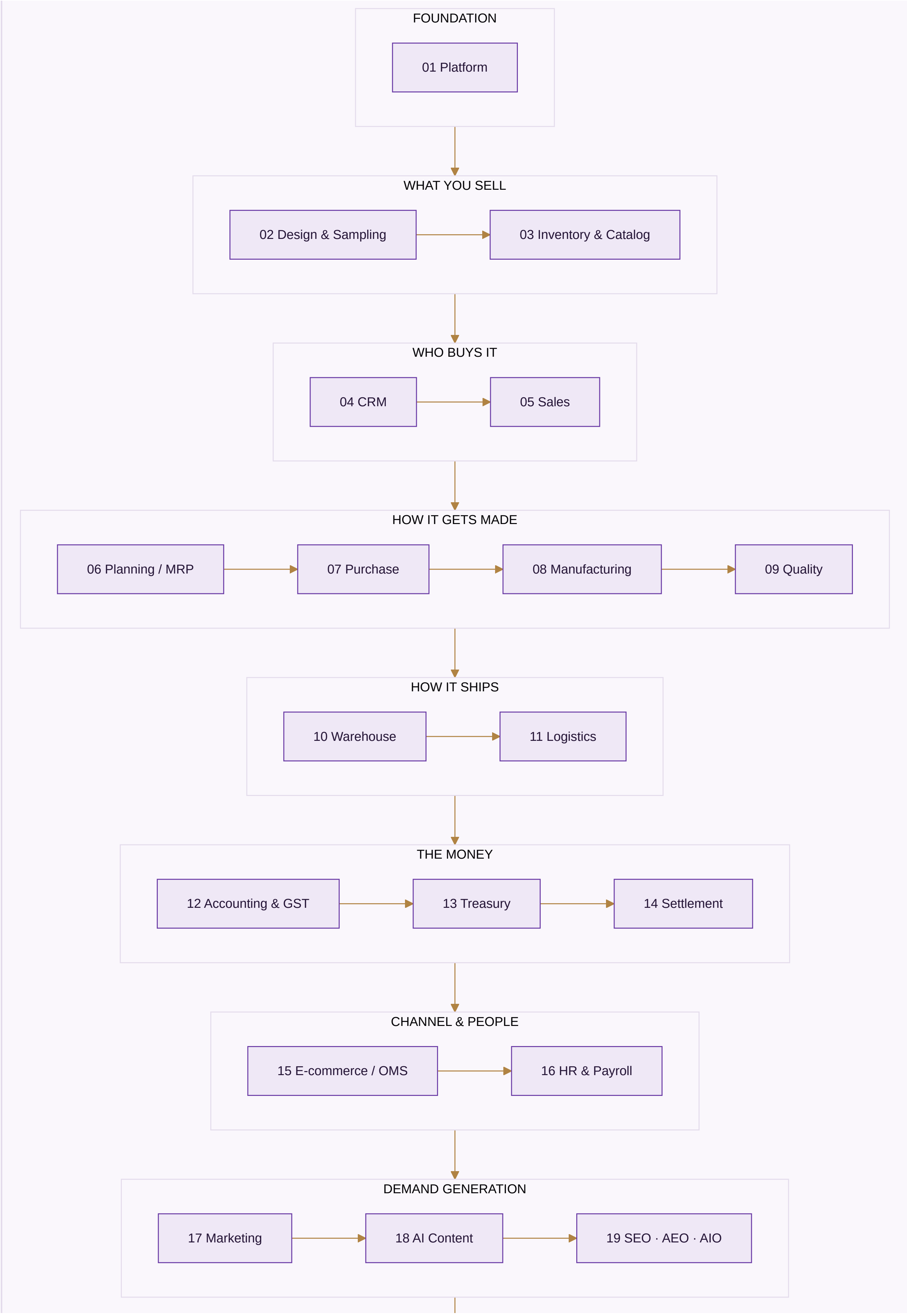
Why this matters more than any feature. The predecessor to this system was a set of separate browser tools, each with its own private storage. A sale recorded in the sales tool never reached the accounting tool. Every integration promise in this document rests on that being fixed: one core, one event bus, and the order the sales module writes is the identical row the accounting module reads.

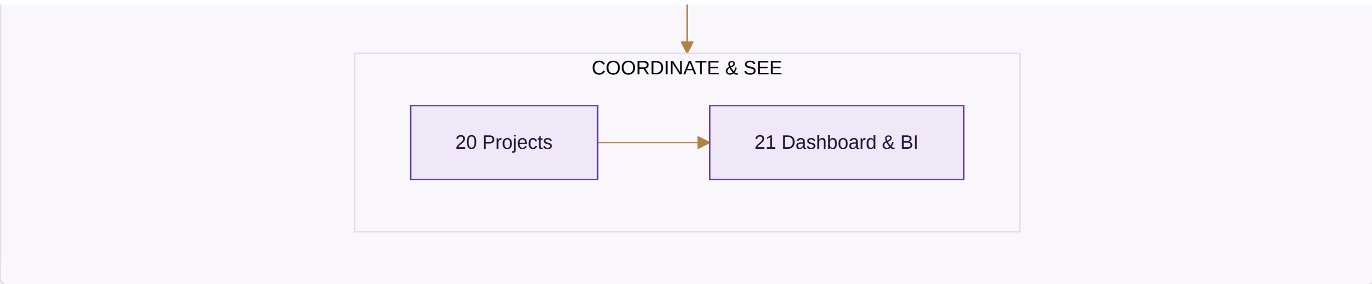
Four rules the core enforces before any module is built

1. **Money is integer paise, never a floating-point number.** Currency arithmetic in floats accumulates error that eventually shows up as a trial balance that will not tie.
2. **Values that change over time are effective-dated** — a salary, a price, a tax rate, a commission. A payroll run for March resolves the salary in force *in March*, not today's. Zero rows in force is an error, never treated as zero.
3. **Nothing is deleted, only deactivated.** A person who leaves is marked inactive; their name is attached to years of earnings, approvals and audit rows that must still resolve.
4. **The audit trail cannot be switched off.** Every edit records who, when, and the value before and after, kept for eight years as the MCA rule requires. It is not a setting that defaults to on — there is no switch, because an audit trail with an off switch is one that gets silenced exactly when it matters.

A3 · THE MODULE MAP — 21 MODULES

Every module lives inside one application: one login, one company switcher, one data core. The numbering below is the build order, and the build order is dependency order — a module is only built once everything it needs already exists.

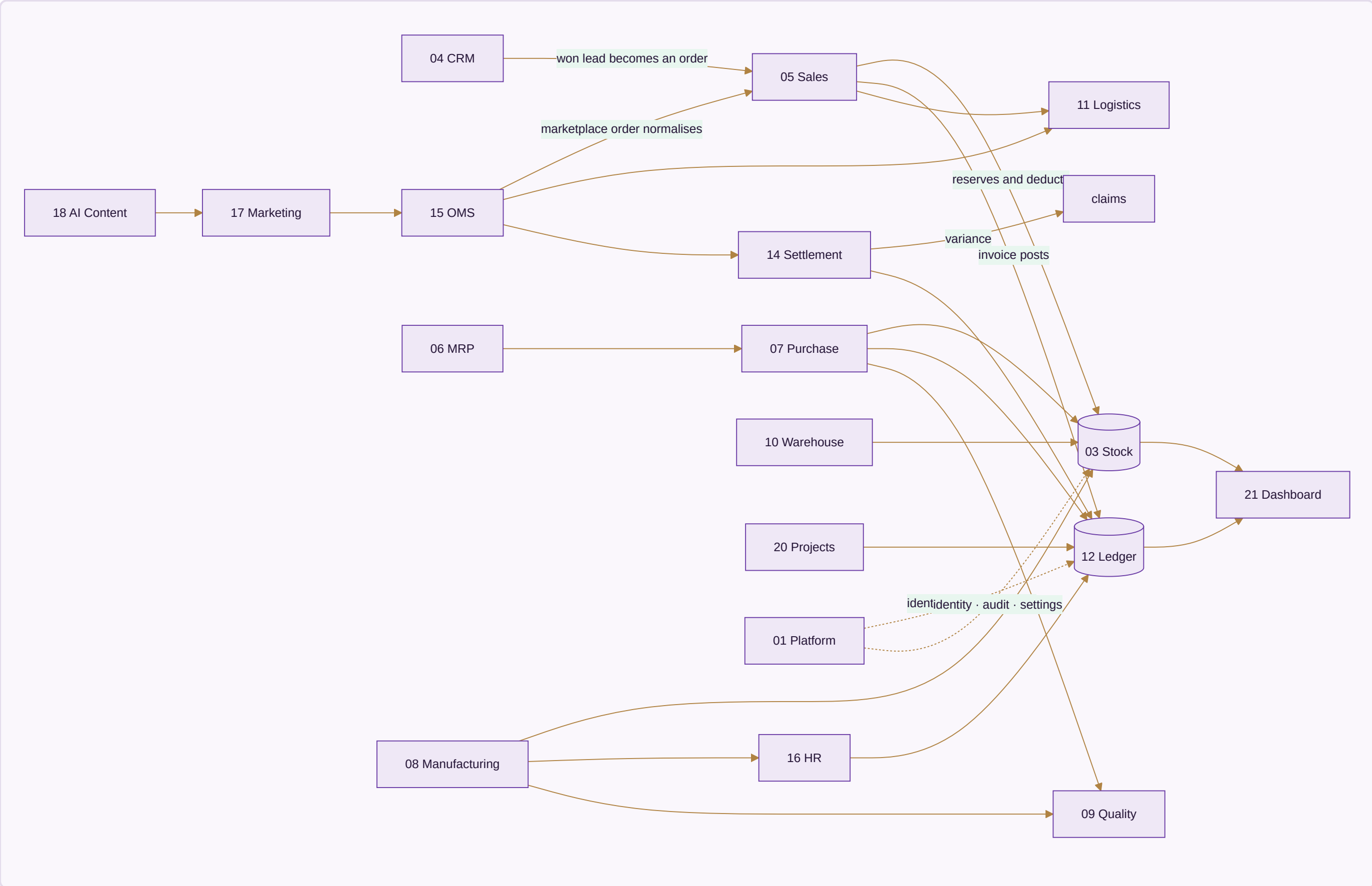

FOUNDATION
01 Platform
WHAT YOU SELL
02 Design & Sampling
03 Inventory & Catalog
WHO BUYS IT
04 CRM
05 Sales
HOW IT GETS MADE
06 Planning / MRP
07 Purchase
08 Manufacturing
09 Quality
HOW IT SHIPS
10 Warehouse
11 Logistics
THE MONEY
12 Accounting & GST
13 Treasury
14 Settlement
CHANNEL & PEOPLE
15 E-commerce / OMS
16 HR & Payroll
DEMAND GENERATION
17 Marketing
18 AI Content
19 SEO · AEO · AIO



#	Module	Apps	Built	To build
01	Platform	5	1	4
02	Design & Sampling	2	0	2
03	Inventory & Catalog	4	0	4
04	CRM	4	3	1
05	Sales	7	5	2
06	Planning & Requirements (MRP)	3	0	3
07	Purchase	3	2	1
08	Manufacturing	4	0	4
09	Quality & Compliance	2	0	2
10	Warehouse	3	0	3
11	Logistics	5	0	5
12	Accounting & GST	9	0	9
13	Treasury & Financial Planning	3	0	3
14	Settlement	3	0	3
15	E-commerce / OMS	11	2	9
16	HR & Payroll	4	0	4
17	Marketing	7	0	7
18	AI Content Engine	5	0	5
19	SEO, AEO & AIO	3	0	3
20	Projects & Collaboration	6	0	6
21	Dashboard & BI	5	3	2
	Total	**98**	**16**	**82**

A4 · MODULE-TO-MODULE WIRING

Every arrow here is a real, required data flow — not an aspiration. This is the fabric that turns 21 modules into one system.



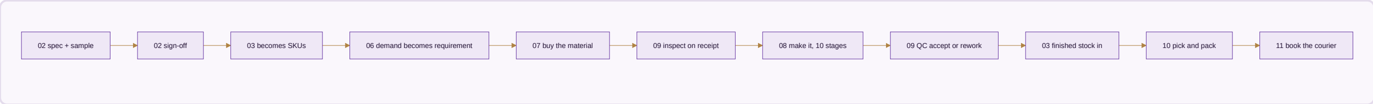
Reading the important arrows.

1. **CRM → Sales.** A won lead becomes a sales order carrying the customer, their price list and their credit tier — nothing re-typed.
2. **Sales → Inventory.** The order reserves and then deducts the single stock number, so the last piece sold anywhere disappears everywhere in the same instant.
3. **Sales → Accounting.** The invoice posts Dr Debtor/Bank, Cr Sales plus output GST, with CGST+SGST or IGST decided from the two GSTINs' state codes — never picked from a dropdown.
4. **OMS → Settlement → Accounting.** Payouts reconcile per line; commission, fees, TCS and TDS post to the books; any variance opens a claim with its evidence.
5. **Purchase → Inventory + Accounting.** The GRN adds stock; the three-way match gates the payable; the GST becomes input credit.
6. **Manufacturing → Inventory + HR + Accounting.** Production adds finished stock, computes karigar piece-rate earnings, posts wages, and rolls cost per piece into design profit.
7. **Accounting → Dashboard.** Every dashboard figure is a query on the ledger, never a separate counter. This is the integrity rule the whole system exists to keep.

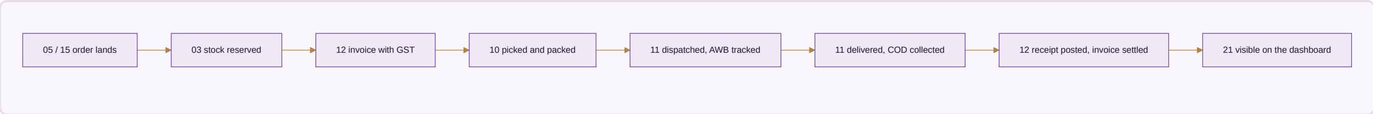
A5 · THE FIVE END-TO-END FLOWS

Four of these are the journeys the business actually runs on. Each crosses many modules, which is exactly the point — no single module completes any of them alone.

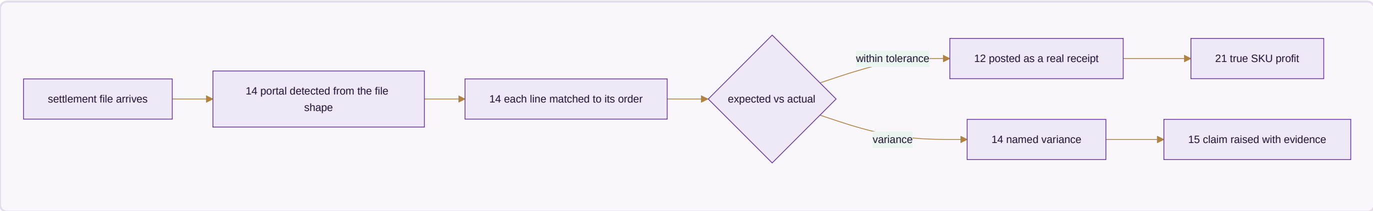
Flow 1 · Design to dispatch



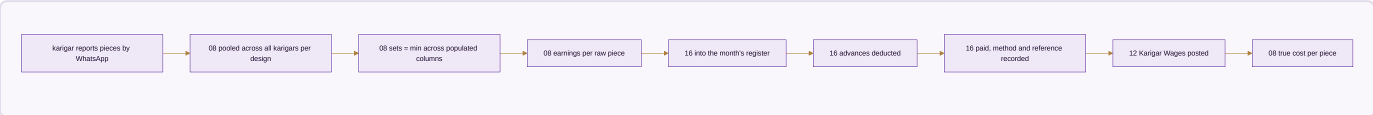
Flow 2 · Order to cash



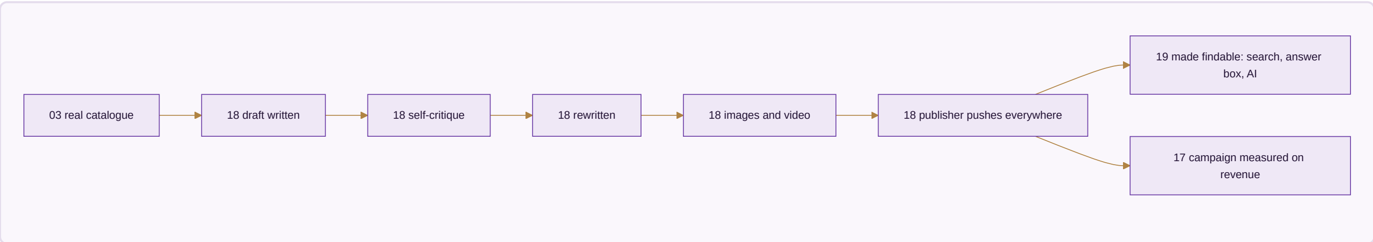
Flow 3 · Settlement to books



Flow 4 · Karigar to payroll



Flow 5 · Content to published



A6 · WHY THIS BUILD ORDER

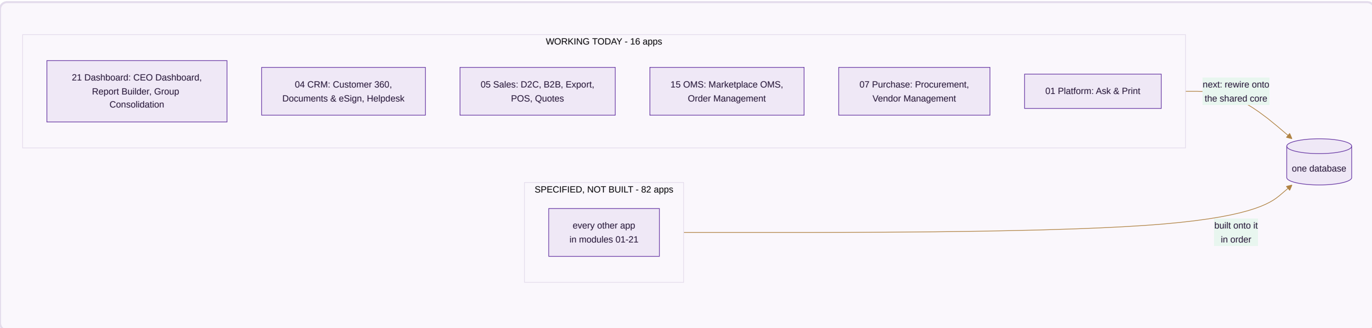
The order is not a preference. Each module needs something the previous one produces, and building out of order means building against data that does not exist yet.

Four decisions worth explaining, because they differ from the obvious arrangement.

- **Platform is first, not last.** Identity, permissions and the audit trail are what every other module writes through. Retrofitting an audit trail means going back through every module.
- **Design & Sampling comes before Inventory.** A product must be specified and approved before it can exist as a catalogue record. Building the catalogue first means inventing styles with no real specification behind them.
- **Quality is its own module, not a step inside Manufacturing.** Rejecting goods at receipt and rejecting goods on the floor are the same discipline, and the certificates that prove a standard is followed belong beside the inspections that back them.
- **Dashboard is last.** It has nothing true to show until the other twenty produce real records. A dashboard built first can only display invented figures.

What a module does when a later module is not built yet. It is finished against the data that exists on its day, and shows more as later modules come online. Module 21's dashboard is real from the moment it is built and grows richer as Module 12's ledger fills.

A7 · THE HONEST STATE TODAY



What "built" means here. Sixteen single-file apps that open in a browser, carry their own self-tests, and pass a full click-through audit with zero console errors in both editions. That is verified, not claimed.

What is honestly not done. Those sixteen apps still run on their own storage. The first work of each module is rewiring its built apps onto the shared core so they read and write the same records as everything else. Until that happens they are good tools, not yet one system.

PART II — THE 21 MODULES

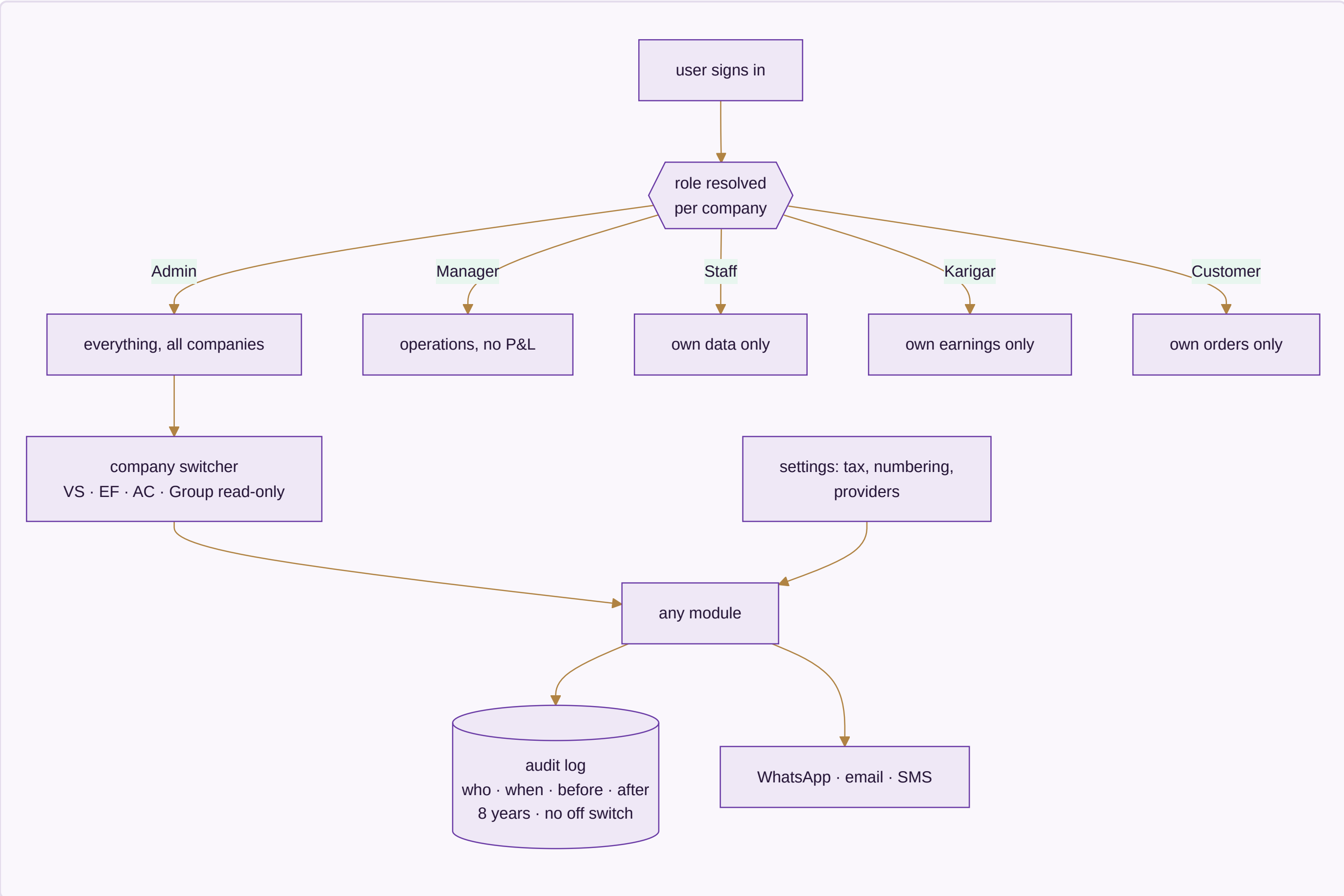
Each module below carries: what it is, a diagram of how it functions, its apps with an honest built or spec mark, the data it owns, the rules that actually govern it, what it reads and writes, and the condition that decides it is finished.

MODULE 01 · PLATFORM

The spine every module runs on

What it is. Not a module you open — the layer underneath the other twenty. Who can see what, how the business is configured, and an unalterable record of everything that ever happened. It is built first because every module above it writes through it, and an audit trail added later is an audit trail with a hole in it.

How it works



The apps (5)

App	State	What it does
Identity, Settings & Audit	SPEC	Users, per-company per-role permissions, company switcher, tax and numbering setup, provider config with an integration-health view, and the browser over the audit trail
Ask & Print	**BUILT**	Ask from a phone — a ledger, a bill, today's packing slips — and get a PDF back, or print at the office with nothing plugged into the phone
Communications	SPEC	WhatsApp command console, broadcasts, email and SMS, and the scheduled jobs that carry a nudge without anyone remembering to send it
Data Privacy & Consent	SPEC	Consent captured where it is given and honoured downstream; retention and erasure tracked as the two different policies they are
Payment Data Scope	SPEC	The written statement of which systems ever see a card or bank credential — and which never do

What it owns. `companies` · `users` · `user_companies` · `audit_log` · `integration_errors` · `settings_environment` · `whatsapp_messages` · `whatsapp_broadcasts` · `email_campaigns` · `notifications` · `consents`

The rules that matter

- Permissions are a matrix — per company, per role — not one global level. A group of sister companies is exactly where a single global level leaks data across a boundary it should not cross.
- The audit trail has no off switch. Eight years, before-and-after values, MCA rule.
- Every capability runs through at least three interchangeable providers. **A provider named as the source of a figure is a bug** — providers move messages and money; the ledger originates numbers.
- Broadcasts go through the official API, warm up at 200/day, and honour a STOP keyword.
- **This system never asks anyone for a marketplace, bank or account password.** A tool that asks for those has become the thing it should protect its users from.

Reads ← every module · **Writes** → every module

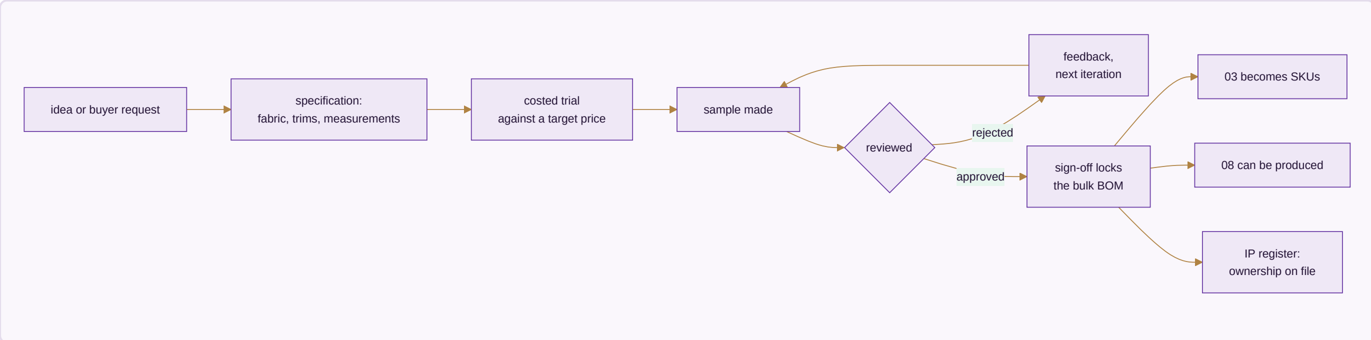
Done when. A karigar sees only their own earnings, an admin switches all three companies and every figure changes with the company, and every edit made during the test is already in the audit browser.

MODULE 02 · DESIGN & SAMPLING

A style exists on paper before it exists as stock

What it is. Where a product moves from a first idea to something the business can actually make and sell — specification, sample rounds, costed trials, sign-off — before it is ever entered as a catalogue record. Building this before Inventory means the catalogue is never populated with styles that have no real specification behind them.

How it works



The apps (2)

App	State	What it does
PLM & Development	SPEC	Specification → sample rounds → costed trials → sign-off, every version kept, so last season's costing is still there
Design / IP Register	SPEC	Trademark and copyright status, the date a design was first shown, and a flag when a near-identical listing appears elsewhere

What it owns. `designs` (the development record) · `samples` · `sample_iterations` · `design_ip_register` · `tech_packs`

The rules that matter

- Every version is kept, never overwritten. A design's cost history is the evidence behind this season's price.
- Sign-off is the event that locks the bulk bill of materials. Sample BOMs and bulk BOMs carry different wastage percentages and are not the same record.
- A design with no ownership record on file is a design nobody can defend — which matters most for exactly the designs Module 18 is built to promote at scale.

Reads ← CRM · **Writes** → Inventory & Catalog · Manufacturing

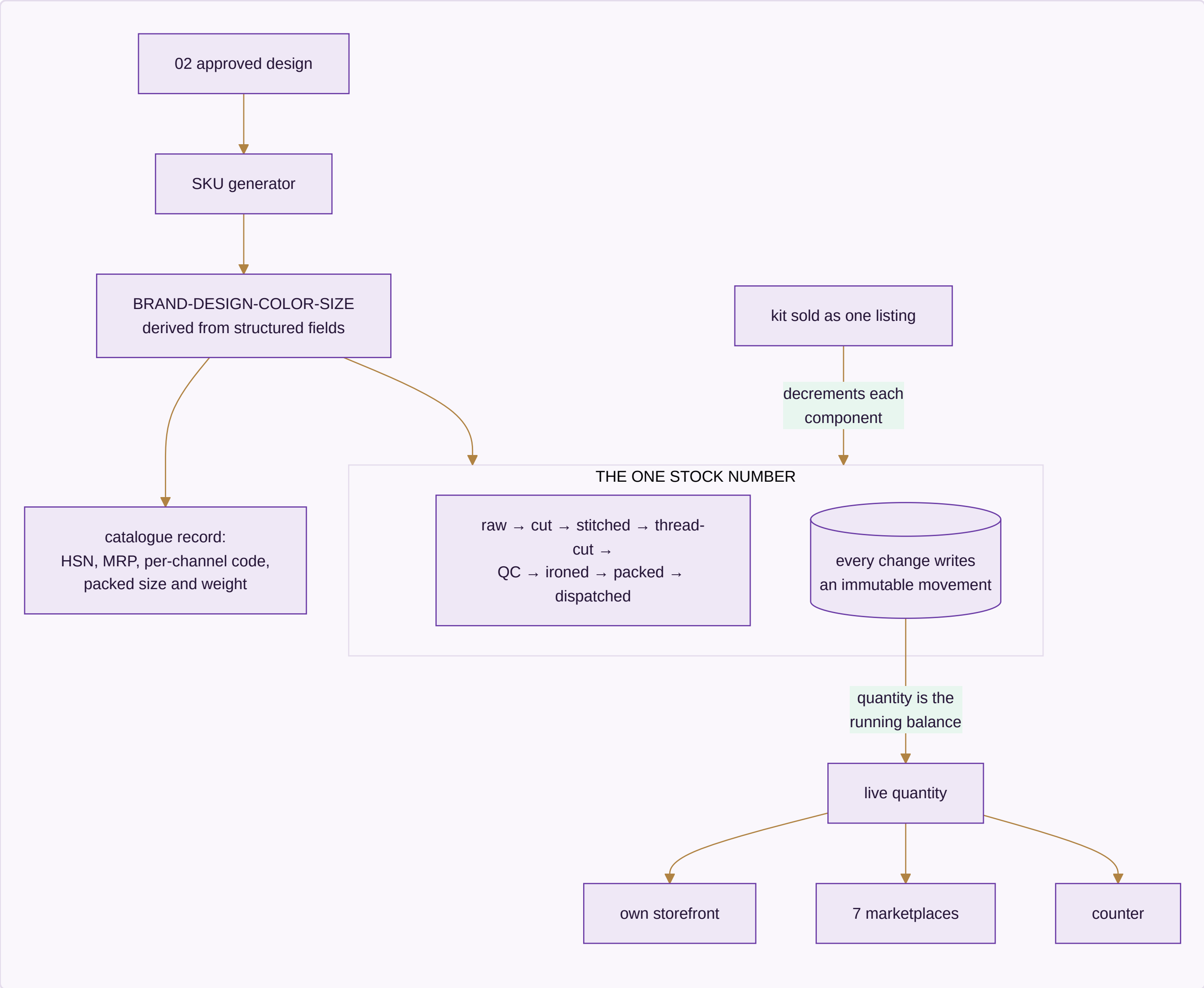
Done when. A design goes idea → sample → rejection → second sample → approval, and only the approved version can generate SKUs.

MODULE 03 · INVENTORY & CATALOG

One number everyone trusts

What it is. The most important number in the system: one quantity per SKU, per location, per stage — read and written by every other module. And one product record that every channel lists from.

How it works



The apps (4)

App	State	What it does
Stock	SPEC	Live quantity by SKU, location and stage, with reorder alerts, batches and dead-stock ageing
Catalog / PIM	SPEC	One product record with each channel's own code mapped to yours, and the packed size and weight that decide courier rate and settle weight disputes
Kit & Combo SKU	SPEC	A sellable SKU made of component SKUs; selling it decrements every component
Master-Data Hygiene	SPEC	Duplicate detection and merge across customers, vendors and designs

What it owns. `designs` · `colors` · `sizes` · `items` · `item_aliases` · `kit_items` · `stock` · `stock_movements` · `batches` · `opening_stock` · `hsn_codes` · `gst_rates` · `locations`

The rules that matter

- **Movements are the ledger; the quantity is its running balance.** Every transition writes an immutable row, so "how did we get to 4?" is always answerable.
- Stock is event-driven and **not per channel**. The last piece sold on one marketplace leaves every other in the same instant — not three hours later as a cancellation, because cancellations are what a seller rating is lost to.
- The SKU string is *derived* from structured fields. Search, grouping and analytics use the fields, never substring-matching on the string.
- Negative stock is a data fault, not a state the business can be in. An issue beyond what exists is refused and recounted.
- A wrong return is dead stock and is **never added back**.
- Stock valuation sets the balance-sheet figure — the two must be equal, or one of them is wrong.

Reads ← Design & Sampling, every module · **Writes** → every module

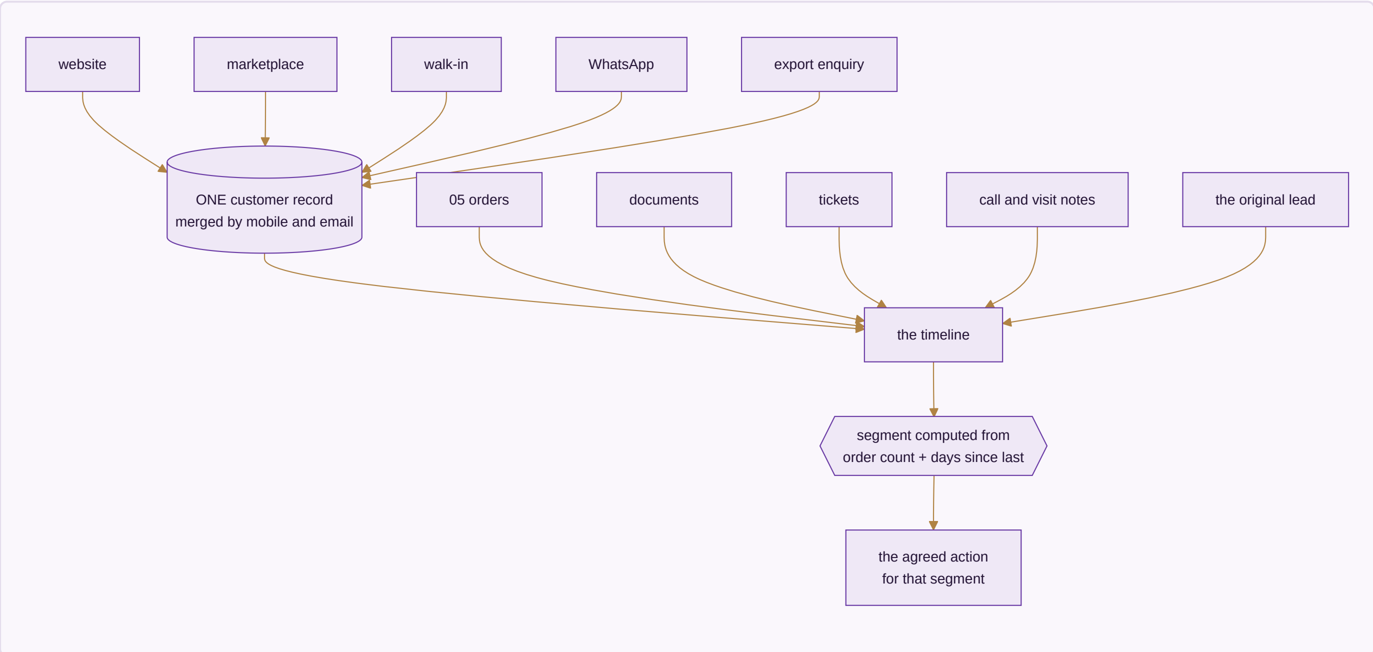
Done when. Stock is one number across every channel, a kit sale decrements all components, and stock valuation equals the balance sheet.

MODULE 04 · CRM

Know every customer completely — and answer them fast

What it is. One record per customer carrying every lead, order, return, document and conversation, whichever channel it arrived on. Whoever picks up the next question can already see everything that came before it.

How it works



The apps (4)

App	State	What it does
CRM & Customer 360	**BUILT**	Lead to won, then the full lifetime — orders, returns, value, and what to offer next, on one screen
Documents & eSign	**BUILT**	Every agreement and certificate filed against the record it belongs to; send for signature and the signed copy files itself back
Helpdesk & Live Chat	**BUILT**	Questions become tickets tied to the order they are about, with the whole history already open
Forms & Feedback (NPS)	SPEC	Post-delivery feedback attached to the *design*, so a complaint-prone design surfaces as a pattern

What it owns. `customers` · `customer_addresses` · `customer_interactions` · `customer_lifecycle_events` · `loyalty_ledger` · `documents` · `tickets` · `nps_responses`

The rules that matter

- Nobody tags anybody. A customer's segment is computed from two facts only — how many orders and how long since the last — so a buyer moves segment by themselves the moment they buy or go quiet. Hand-tagged segments are always six months out of date.
- D2C, B2B, export, walk-in and WhatsApp merge to one customer by mobile and email. Marketplace customers stay separate because the channel does not share their details, but tie by pattern.
- The pipeline advances Lead → Qualified → Quoted → Negotiation and stops there; won or lost is a separate deliberate act.
- Feedback attaches to the design, not only the buyer — that is what turns a scatter of complaints into a fault Manufacturing can fix.

Reads ← every module · **Writes** → Sales · E-commerce/OMS · Marketing

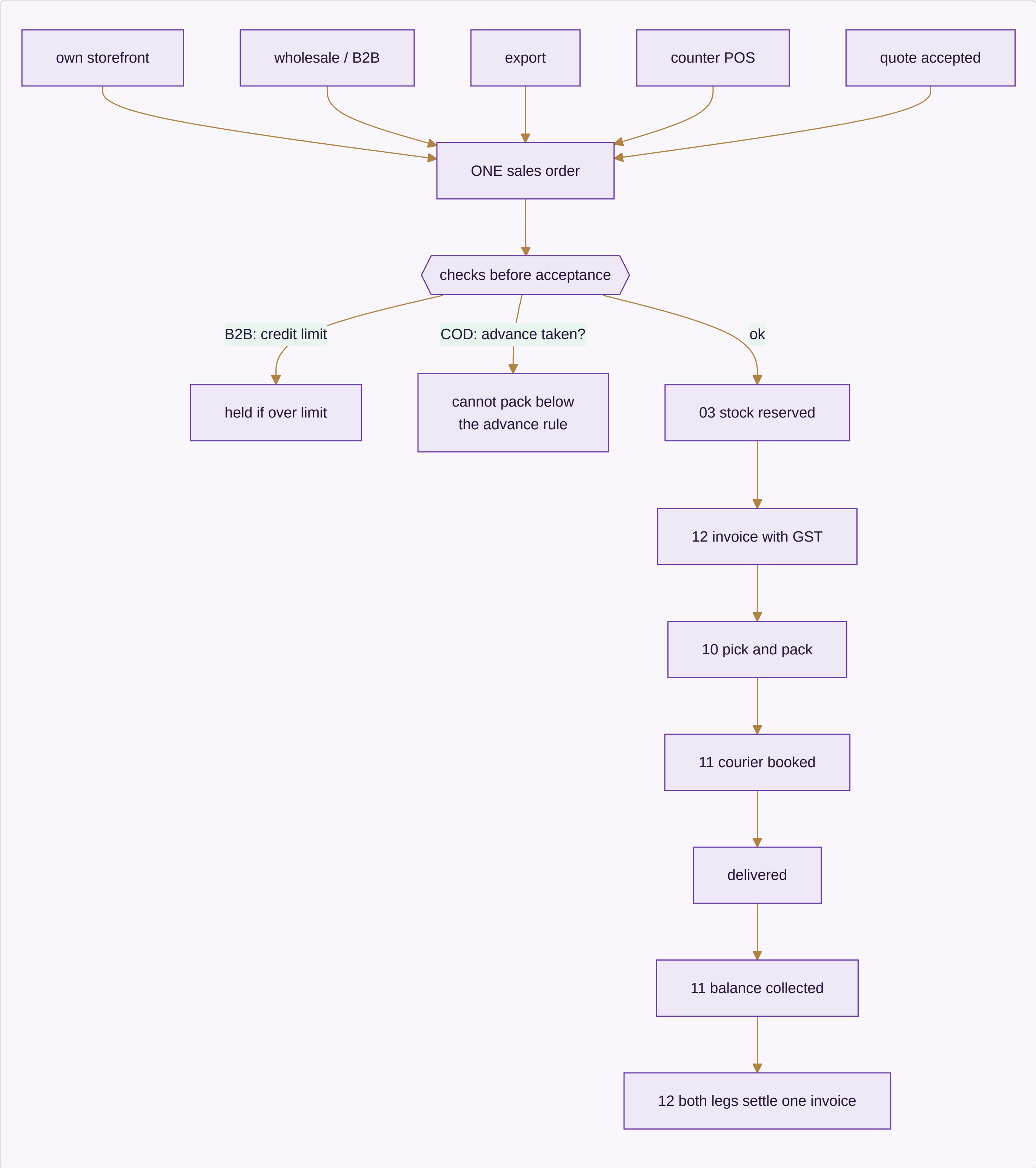
Done when. One customer's whole cross-channel history is on one screen, and crossing an order threshold moves their segment without anyone touching it.

MODULE 05 · SALES

Every way you sell, one order book — to the doorstep

What it is. Counter, wholesale, export and your own website all writing to the same order and drawing on the same stock number. A sale is not finished when it is billed — it is finished when it is delivered and the cash-on-delivery money is in.

How it works


own storefront
wholesale / B2B
export
counter POS
quote accepted
ONE sales order
checks before acceptance
B2B: credit limit
COD: advance taken?
ok
held if over limit
cannot pack below the advance rule
03 stock reserved
12 invoice with GST
10 pick and pack
11 courier booked
delivered
11 balance collected
12 both legs settle one invoice

The apps (7)

App	State	What it does
D2C Sales	**BUILT**	Own-storefront orders cart to dispatch, with loyalty and partial COD
B2B & Credit	**BUILT**	Wholesale orders with credit limits, tier pricing and outstanding ageing
Export	**BUILT**	Commercial invoice, packing list, LUT bond and IGST-refund tracking
POS	**BUILT**	Counter billing drawing on the same stock as the website
Quotes & Proforma	**BUILT**	Send a quote, convert it to a confirmed order in one step
Couriers & AWB	SPEC	Compare couriers on the order, print the label, follow the AWB to the door
Subscriptions	SPEC	A schedule that raises its own invoice and follows up when a payment fails

What it owns. `sales_orders` · `sales_order_items` · `invoices` · `invoice_items` · `b2b_orders` · `b2b_credit_ledger` · `export_orders` · `customization_orders` · `subscriptions`

The rules that matter

- **A COD order cannot be packed until it carries its advance.** A refused COD parcel costs the courier fee both ways and comes back handled; a customer who has paid something almost always accepts. This one rule is the cheapest defence the business has.
- **Partial COD reconciles both legs to one invoice** — the advance taken online and the balance the courier remits.
- Credit is checked *before* acceptance, not discovered later as a bad debt. Reminders at −3 days, +1 day, and a +7-day soft block.
- The delivery date is **derived** — cut-off plus transit from the warehouse that actually holds stock — never typed. An order no warehouse can serve gets no date at all, because a promise nobody can keep is worse than no promise.
- Export lines are zero-rated under LUT; FX difference between billing rate and realisation rate posts to FX gain/loss.

Reads ← Inventory & Catalog · CRM · Warehouse · Logistics · **Writes** → Inventory & Catalog · Accounting & GST · Warehouse · Logistics

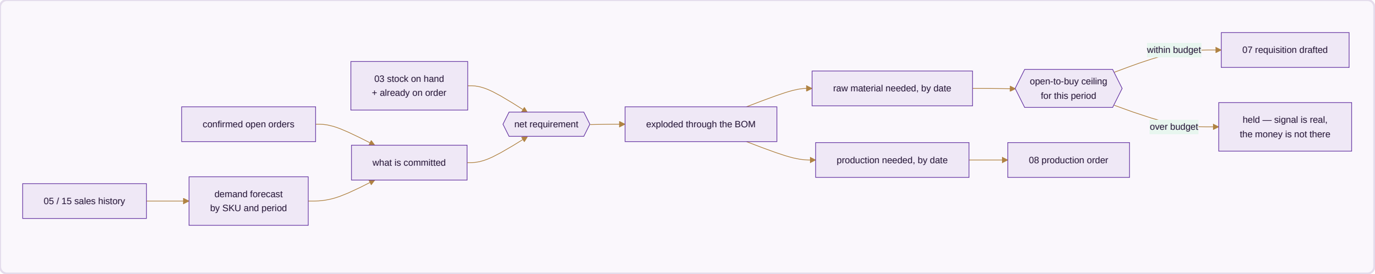
Done when. A storefront order appears within a minute with stock reserved and an invoice raised, and a partial-COD order reconciles both legs by itself.

MODULE 06 · PLANNING & REQUIREMENTS (MRP)

Turn what is selling into what to buy and make

What it is. The module that sits between demand and supply. Confirmed orders and sales history become a requirement before Purchase buys anything or Manufacturing starts anything — otherwise both are guessing.

How it works



The apps (3)

App	State	What it does
Demand Forecast & Signal	SPEC	What sold, by SKU and period, turned into a short-term forecast — the number every downstream requisition is measured against
Requirement Explosion (MRP run)	SPEC	Demand exploded through the bill of materials into what to buy and what to make, and by when
Open-to-Buy / Budget Ceiling	SPEC	A spending ceiling per period that a demand signal alone cannot exceed

What it owns. `demand_forecast` · `mrp_runs` · `mrp_requirements` · `open_to_buy_budgets`

The rules that matter

- Net requirement is demand minus stock on hand minus what is already on order. Buying against gross demand is how a business ends up holding two seasons of the same fabric.
- Every requisition traces back to the run that produced it, and that run back to the demand behind it. A purchase order whose only justification is "stock looked low" is exactly what this module exists to eliminate.
- **The budget ceiling is a hard guardrail, not a warning.** A genuine demand signal can still commit more money than the business decided to risk this season, and without a ceiling it will.

Reads ← Sales · E-commerce/OMS · Inventory & Catalog · **Writes** → Purchase · Manufacturing

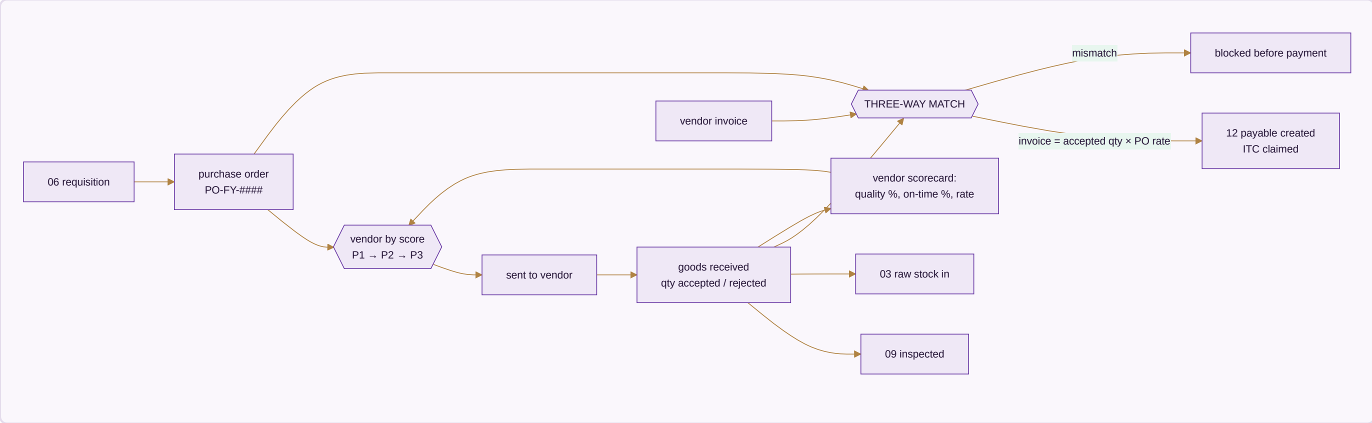
Done when. An MRP run turns a month of real demand into requisitions and production orders that each trace back to the orders that caused them, and the ceiling blocks the run that exceeds it.

MODULE 07 · PURCHASE

Nothing over-billed gets paid

What it is. The buy side end to end — mills, dyers, job workers, packing suppliers — and the control that stops you paying for goods you rejected.

How it works



The apps (3)

App	State	What it does
Procurement	**BUILT**	Requisition → PO → GRN, with a strict three-way match before any bill is paid
Vendor Management	**BUILT**	Vendor 360, payables, ageing, a real risk score, and sourcing that follows performance
Insurance Register	SPEC	Cover over stock in transit, stock in the warehouse and product liability, matched against what is actually moving

What it owns. `purchase_requisitions` · `purchase_orders` · `purchase_order_items` · `grn` · `grn_items` · `vendor_invoices` · `three_way_match` · `vendors` · `vendor_materials` · `third_party_services` · `insurance_policies`

The rules that matter

- **The three-way match is the whole point.** You ordered 100 metres, 100 arrived, quality accepted 96 — the bill clears for 96 and not a metre more. The system flags GRN ≠ PO, invoice ≠ accepted × rate, and any invoice arriving before its GRN.
- The vendor scorecard is computed per transaction from real outcomes — accept rate, on-time delivery, rate against market — and it drives the priority order for the next requisition. Sourcing follows performance rather than habit.
- Real stock value moves through purchase and the warehouse continuously; cover that nobody tracks is cover nobody can claim on.

Reads ← Inventory & Catalog · Planning/MRP · Manufacturing · **Writes** → Inventory & Catalog · Accounting & GST · Quality & Compliance

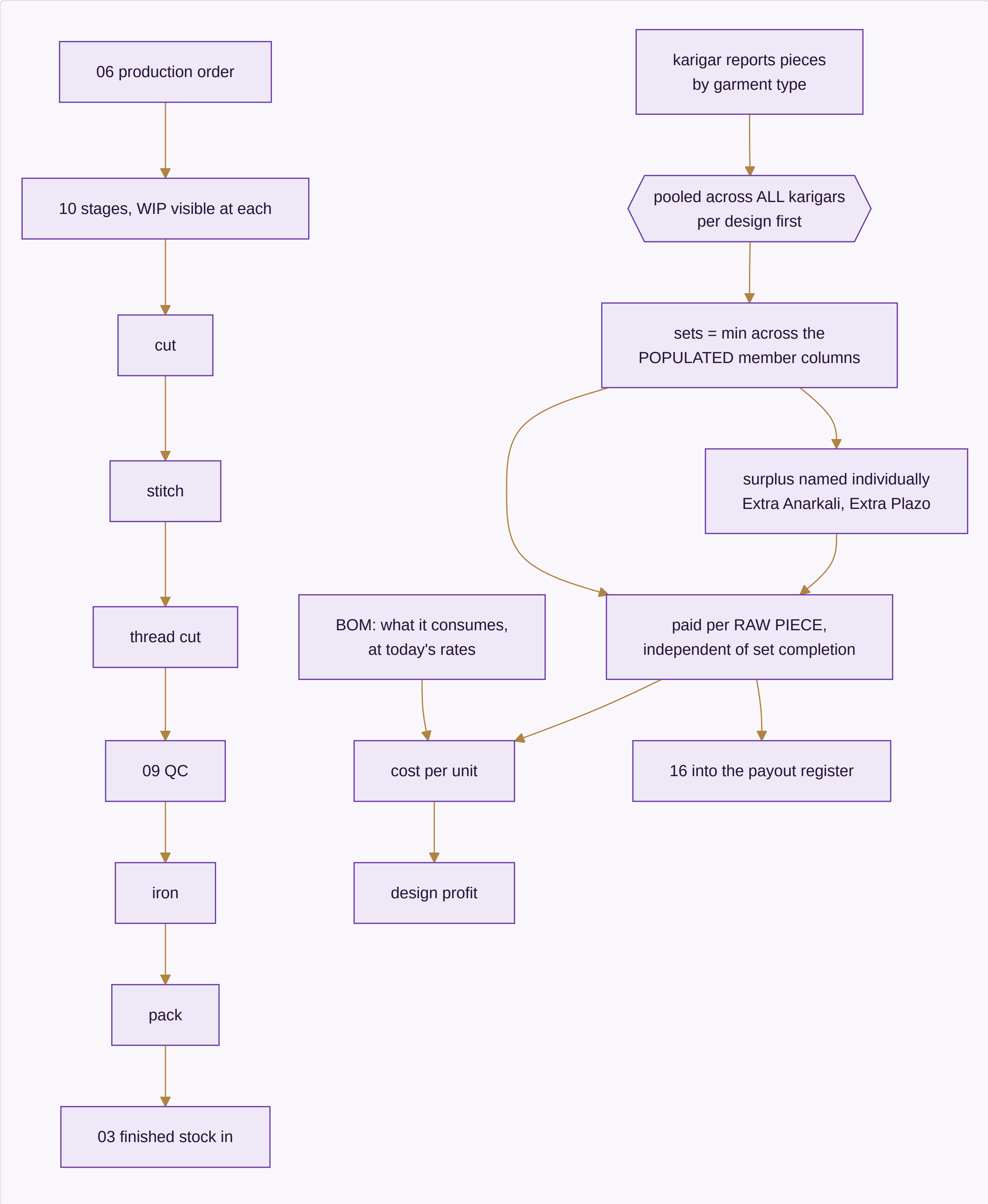
Done when. A vendor invoice for more than was accepted is blocked before payment, and the rejection reason is on the record.

MODULE 08 · MANUFACTURING

Know what a unit really costs to make

What it is. From material in the door to the finished unit — every operation, every worker's earning, and what each design actually cost before it was priced. Design sign-off happens upstream in Module 02; inspection and the compliance record live downstream in Module 09. This module is purely the making.

How it works


06 production order
10 stages, WIP visible at each
cut
stitch
thread cut
09 QC
iron
pack
03 finished stock in
karigar reports pieces by garment type
pooled across ALL karigars per design first
sets = min across the POPULATED member columns
surplus named individually Extra Anarkali, Extra Plazo
paid per RAW PIECE, independent of set completion
BOM: what it consumes, at today's rates
cost per unit
design profit
16 into the payout register

The apps (4)

App	State	What it does
Production Orders	SPEC	Your own stages from first operation to finished goods, with work-in-progress visible at each
Piece-rate & Contractors	SPEC	Output-based pay: pooled set completion, per-unit rates, rework and advances resolved into one payout
BOM & Consumption	SPEC	What each design consumes — fabric, zari, lining, trims, packing — costed at today's rates
Maintenance	SPEC	Machines and tools: what is due for service, what it cost, what stopped while it was down

What it owns. `production_orders` · `production_stages` · `bom` · `bom_items` · `karigar_assignments` · `karigar_reports` · `performance_flags`

The karigar costing rules — the heart of this module

These are settled and already proven in a working engine. They are unusual enough to be worth stating exactly, because getting any one of them wrong changes what a person gets paid.

1. **23 garment columns collapse to 13 set types.** A set is a named combination, not an arbitrary grouping.
2. **Pool across all karigars per design first, then apply the rule.** Pooling after applying the rule produces a different, wrong answer.
3. **Sets = the minimum across the *populated* member columns of that set type.** A column nobody worked on is not a zero that drags the set count to nothing — it is simply not part of the calculation.
4. **Extras are named individually** — Extra Anarkali, Extra Plazo — never a generic leftover bucket, and there is **no combined "Set + Extra" total column**, because that column would double-count.
5. **Cost is per raw piece, independent of set completion.** A surplus piece that completes no set is still work somebody did, and is still paid.
6. **A missing rate posts ₹0 and raises a flag — never a guessed rate.** A guessed rate is a silent error in someone's wages.
7. Alteration earning is alteration hours × ₹100; an alteration caused by the worker's own mistake is ₹0.
8. A performance flag fires when the same person, design and task takes more than 1.2× the previous hours, and asks why by WhatsApp.

The acceptance gate — figures that must reproduce to the rupee

Designs	Karigar units	Sets	Pieces	Total	Flagged
143	29	**25,307**	**59,110**	**₹26,90,062**	5 designs with no rate

A mismatch against these is a bug, not a rounding difference. The existing Python engine is kept as the independent checker: the ported engine must agree with it to the paise.

Reads ← Purchase · Planning/MRP · Design & Sampling · **Writes** → Inventory & Catalog · HR & Payroll · Accounting & GST · Quality & Compliance

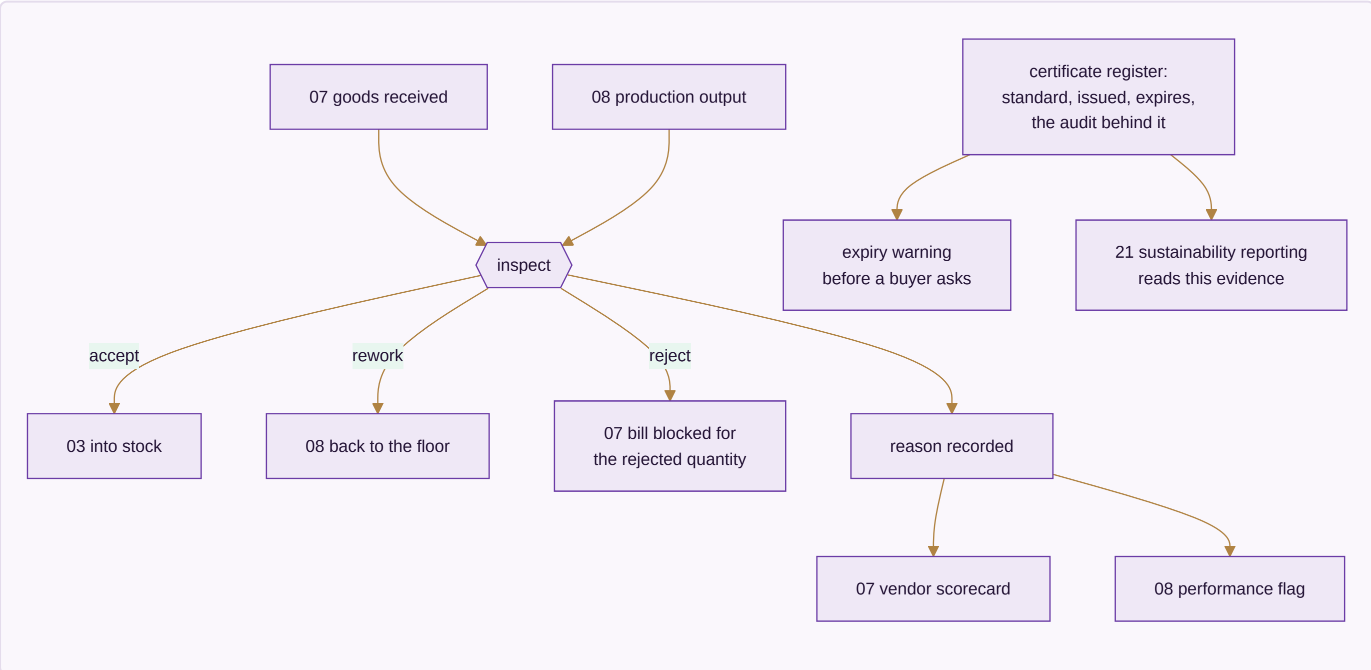
Done when. Three production orders run to completion — self-made, full job work, partial job work — and the acceptance-gate totals reproduce exactly.

MODULE 09 · QUALITY & COMPLIANCE

Certify what was received and what was made

What it is. Inspection used to be one step buried inside Manufacturing. It stands on its own here because rejecting goods at receipt and rejecting goods on the floor are the same discipline — and because the certificates that prove a standard is followed belong beside the inspections that back them up, not scattered across email.

How it works



The apps (2)

App	State	What it does
Quality Control	SPEC	Accept, reject or rework — on goods received and goods made — with reasons that feed the vendor scorecard and the performance flags alike
Certificate & Compliance Register	SPEC	Every standard held, with issue date, expiry and the audit that backs it

What it owns. `qc_records` · `qc_defect_categories` · `compliance_certificates` · `compliance_audits`

The rules that matter

- A rejection is never just a number — it carries a reason, and that reason is what makes the vendor scorecard and the performance flag meaningful rather than decorative.

- The quantity accepted at QC, not the quantity delivered, is what the three-way match pays against. This is the join between this module and the money.
- A certificate about to lapse is visible *before* a buyer asks for it and finds it expired.
- What this register holds is precisely what a sustainability report is later built from — which is why the reporting app in Module 21 reads it rather than collecting evidence separately.

Reads ← Purchase · Manufacturing · **Writes** → Purchase · Manufacturing · Inventory & Catalog

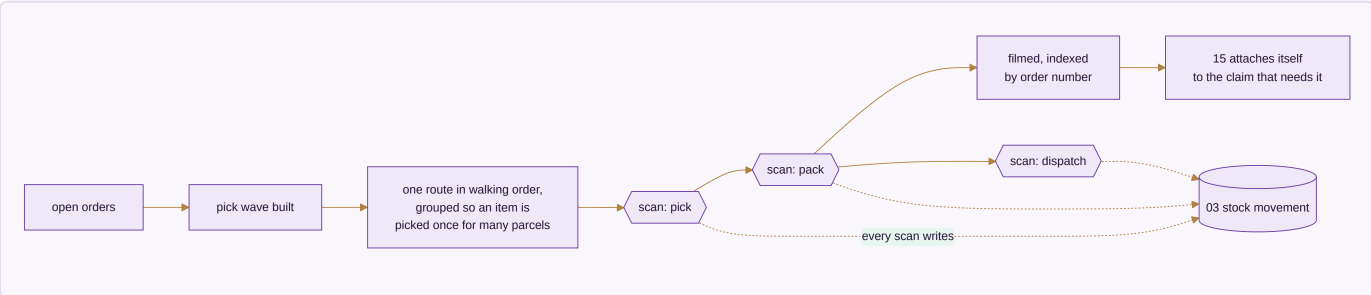
Done when. A rejected receipt blocks payment for exactly the rejected quantity, and an expiring certificate raises a warning before its expiry date.

MODULE 10 · WAREHOUSE

Pick right the first time — and prove what you sent

What it is. Bin-level instructions and barcode scanning so the right item leaves the building and stock stays honest — and a recording of each parcel as it is packed, so an argument about what was in it is settled by footage instead of memory.

How it works



The apps (3)

App	State	What it does
Picking & Bins	SPEC	Pick lists that name the bin, in walking order, so nobody crosses the floor twice
Barcode Operations	SPEC	Scan to pick, pack, dispatch and count from a phone — the same scan whatever channel the order came from
Packing Video	SPEC	Every parcel filmed as it is packed and indexed by order number

What it owns. `bins` · `pick_lists` · `pick_list_lines` · `barcode_scans` · `packing_videos`

The rules that matter

- A picker is never sent to an empty rack — the wave is built from the stock record, not from hope.
- **Every scan writes a stock movement.** That is what keeps the running balance real rather than a number someone updates at the end of the day.

- The day is grouped by product, so one design is picked once for eleven parcels instead of eleven times.
- The footage attaches to the claim by order number automatically. A wrong-item claim answered with the clip is a claim won on evidence rather than argument.

Reads ← Sales · E-commerce/OMS · Inventory & Catalog · **Writes** → Inventory & Catalog · Sales · E-commerce/OMS

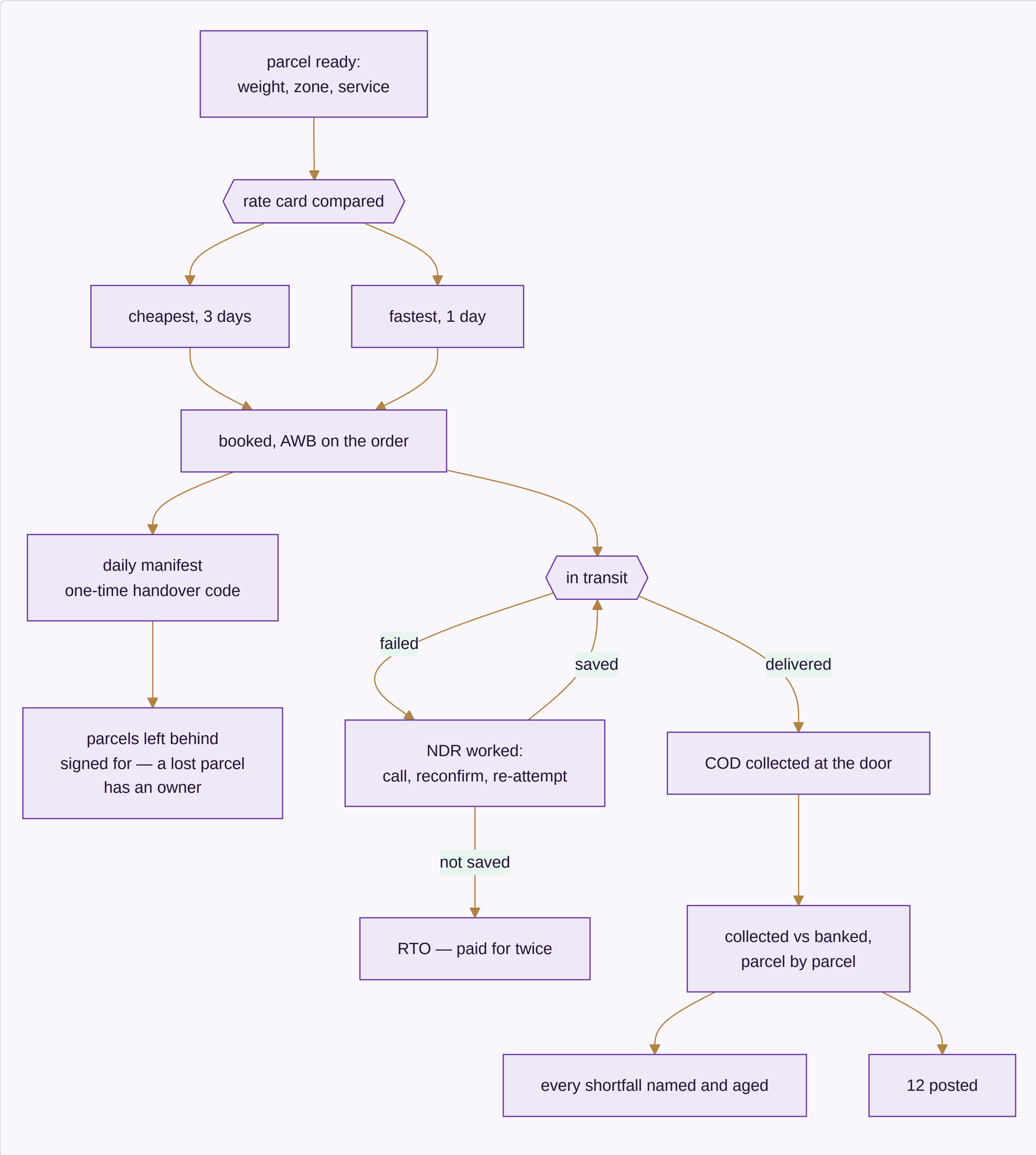
Done when. A parcel is picked from the right bin, filmed, and its clip is already attached when the claim arrives.

MODULE 11 · LOGISTICS

The courier network — rates, failed deliveries and the COD money

What it is. Booking one parcel happens on the order, in Sales. This module is the network behind it: what each courier charges before you pick one, what happens to a delivery that fails, and whether the cash collected at the door actually reached your bank.

How it works


parcel ready:
weight, zone, service
rate card compared
cheapest, 3 days
fastest, 1 day
booked, AWB on the order
daily manifest
one-time handover code
in transit
failed
saved
delivered
parcels left behind
signed for — a lost parcel
has an owner
NDR worked:
call, reconfirm, re-attempt
COD collected at the door
not saved
RTO — paid for twice
collected vs banked,
parcel by parcel
every shortfall named and aged
12 posted

The apps (5)

App	State	What it does
Rates & Zones	SPEC	Every courier's rate card by zone, weight slab and service — cheapest and fastest both known before booking
NDR & RTO Rescue	SPEC	A failed delivery worked while it can still be saved, before it becomes a return you pay for twice
COD Remittance	SPEC	What was collected at the door against what reached the bank, parcel by parcel
Handover & Manifest	SPEC	What went out against what the courier actually took, with a one-time code and a signed record of what was left
Fleet	SPEC	Own vehicles, if any — trips, cost, service due. Optional; most run couriers only

What it owns. `courier_rates` · `shipments` · `ndr_cases` · `cod_remittance` · `manifests` · `vehicles`

The rules that matter

- Both the cheapest and the fastest option are known before booking. Choosing without seeing both is choosing blind.
- **An NDR is worked while it can still be saved.** An RTO costs freight in both directions and returns handled goods — the rescue attempt is almost always cheaper than the return.
- COD is reconciled parcel by parcel, and every shortfall is named and aged. A lump-sum remittance that roughly matches is not reconciliation.
- The manifest carries a signed record of what was *not* taken, so a parcel lost between the packing table and the van has an owner.

Reads ← Sales · E-commerce/OMS · Warehouse · **Writes** → Accounting & GST · Sales · E-commerce/OMS

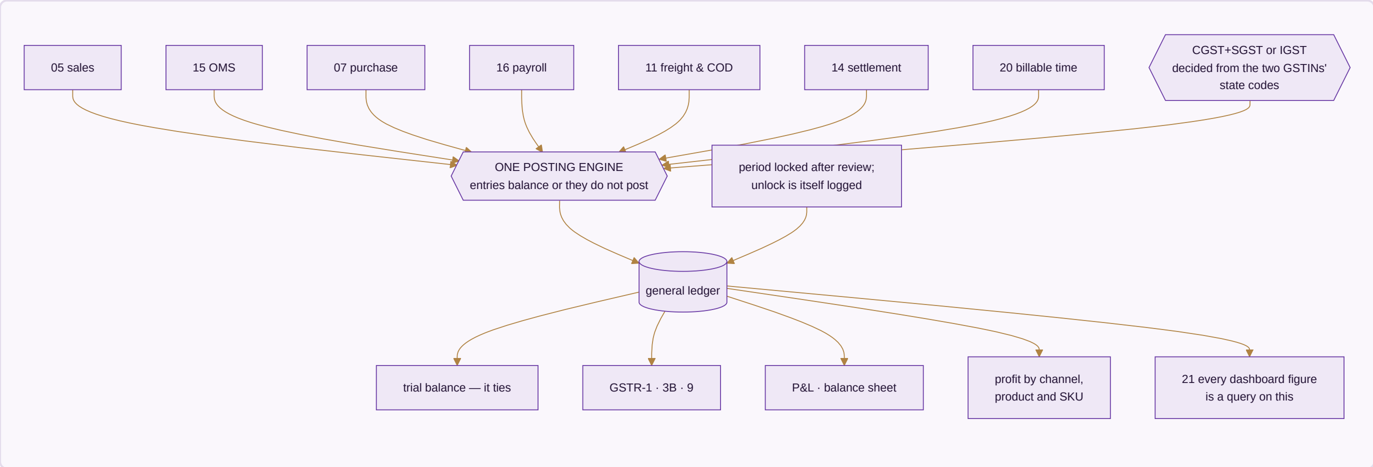
Done when. COD collected reconciles to COD banked parcel by parcel, and an NDR is worked before it turns into an RTO.

MODULE 12 · ACCOUNTING & GST

Books that always balance

What it is. A full double-entry ledger built for Indian compliance — not a tax report bolted onto a spreadsheet. This system keeps the books on its own; no other accounting package is required, ever. Existing packages stay available as connectors for anyone already running one, but no figure in the business is ever *sourced* from them.

How it works



The apps (9)

App	State	What it does
Accounting	SPEC	Chart of accounts, nine voucher types, and the one posting engine every voucher writes through
Invoicing	SPEC	GST tax invoices computed from the lines to the paise, with round-off and e-invoice IRN
Expenses	SPEC	Spend by category with approvals, and bill OCR to save typing
GST & Tax	SPEC	CGST, SGST, IGST, TDS, TCS, input credit and the GSTR returns, filed per registration
ITC Reconciliation	SPEC	Purchases matched against the portal's own GSTR-2A/2B before a return is filed
Receivables, Payables & PDC	SPEC	Payments allocated against named invoices, and post-dated cheques posting on realisation
Fixed Assets & Depreciation	SPEC	The asset register with both straight-line and written-down-value tracked side by side
Year-End Close & Period Lock	SPEC	Carry-forward at year end, and a locked period no backdated edit can touch
Finance Reports	SPEC	P&L, balance sheet, and profit by channel, product and SKU

What it owns. `chart_of_accounts` · `voucher_series` · `journal_entries` · `journal_lines` · `gst_returns` · `gst_input_credit` · `tds_entries` · `tcs_entries` · `bank_accounts` · `bank_transactions` · `fixed_assets` · `depreciation_entries` · `post_dated_cheques` · `bill_allocations` · `period_locks`

The rules that matter

- **One posting engine.** Every voucher writes the ledger through it, and an entry that does not balance does not post — it is refused, not saved half-done.
- **The tax type is derived, never chosen.** Intra-state versus inter-state comes from comparing the two GSTINs' state codes. A hand-picked tax type is a hand-made error.
- Rates default from the HSN and are **versioned by effective date**, so an invoice from last year keeps last year's rate after a rate change.

- Input credit is reconciled against the government's own data before GSTR-3B. Without it, a filing is a guess about how much credit you may legitimately claim.
- **Round-off posts to its own ledger account** — never absorbed into the sale amount, because absorbing it corrupts the GST calculation underneath.
- A closed period locks. No backdated edit without an admin unlock, and the unlock is itself logged.

Build order inside this module — fixed by the nature of the thing: posting engine and audit first, wrapping every write from day one; then sales and purchase invoices, verified against the trial balance; then the other seven voucher types; then year-end close and period lock; then GST returns and 2A/2B reconciliation.

Reads ← every module · **Writes** → Finance Reports · Treasury

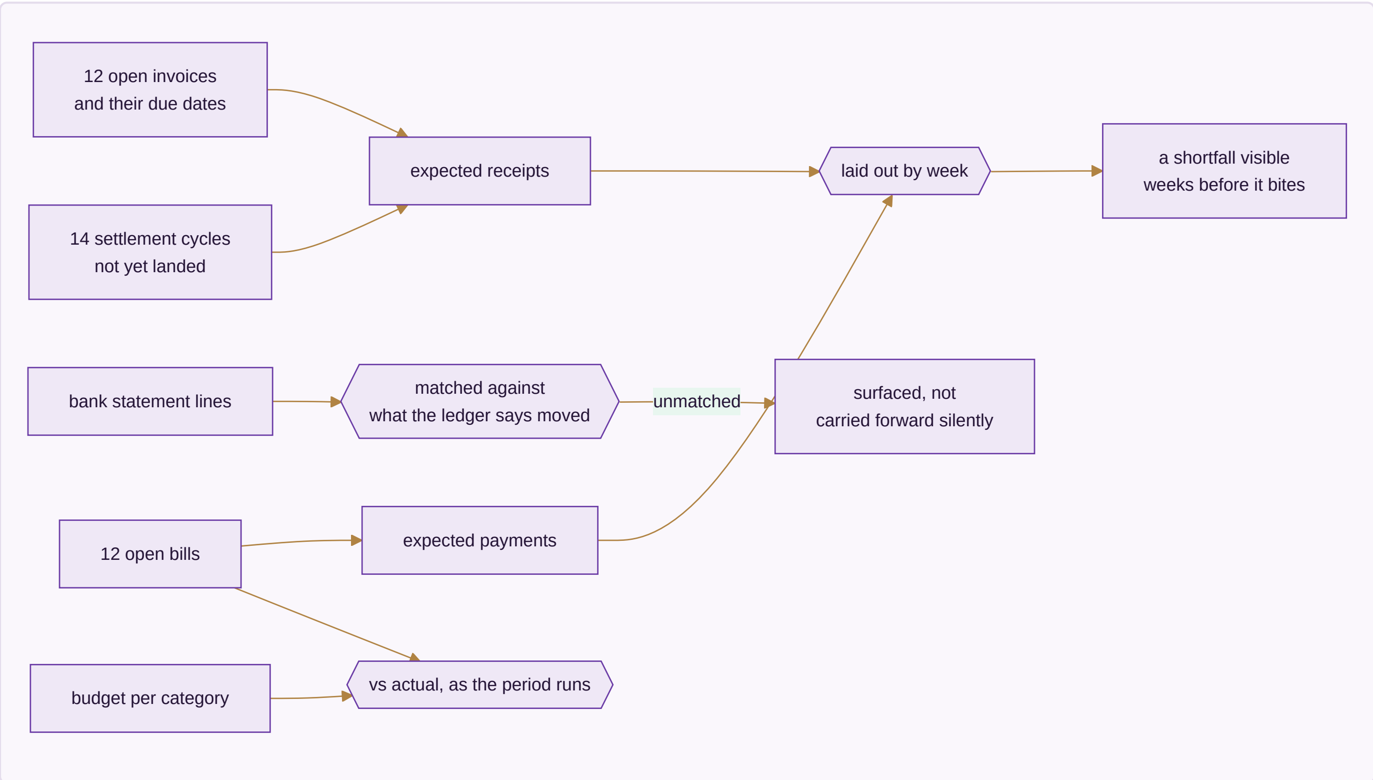
Done when. One month of books closes cleanly, the trial balance ties, and GSTR-1 and GSTR-3B generate and verify.

MODULE 13 · TREASURY & FINANCIAL PLANNING

Know what cash is coming, not just what already arrived

What it is. Accounting records what happened. This module is about what happens next — how much cash is genuinely expected and when, and whether spending against a budget is on track while there is still time to act.

How it works



The apps (3)

App	State	What it does
Cash Flow Forecast	SPEC	Expected receipts and payments by week, drawn from open invoices and bills rather than typed in
Banking & Reconciliation	SPEC	Statement lines matched against the ledger's own record, with anything unmatched surfaced
Budget vs Actual	SPEC	A budget per category and period, tracked against real spend while the period is still running

What it owns. `cash_forecasts` · `bank_reconciliation_sessions` · `budgets` · `budget_lines`

The rules that matter

- The forecast is **derived from open invoices and bills**, not typed in. A forecast someone maintains by hand is out of date the day after it is written.
- Marketplace payouts that have not landed yet are part of expected cash — that is what makes the forecast honest for a business selling on seven channels.
- An unmatched bank line is surfaced, never quietly carried forward. Carried-forward differences are how a reconciliation becomes fiction.
- Budget variance is shown **while the period runs**. A variance discovered after close is history; the only thing left to do with it is explain it.

Reads ← Accounting & GST · Sales · Purchase · **Writes** → Accounting & GST

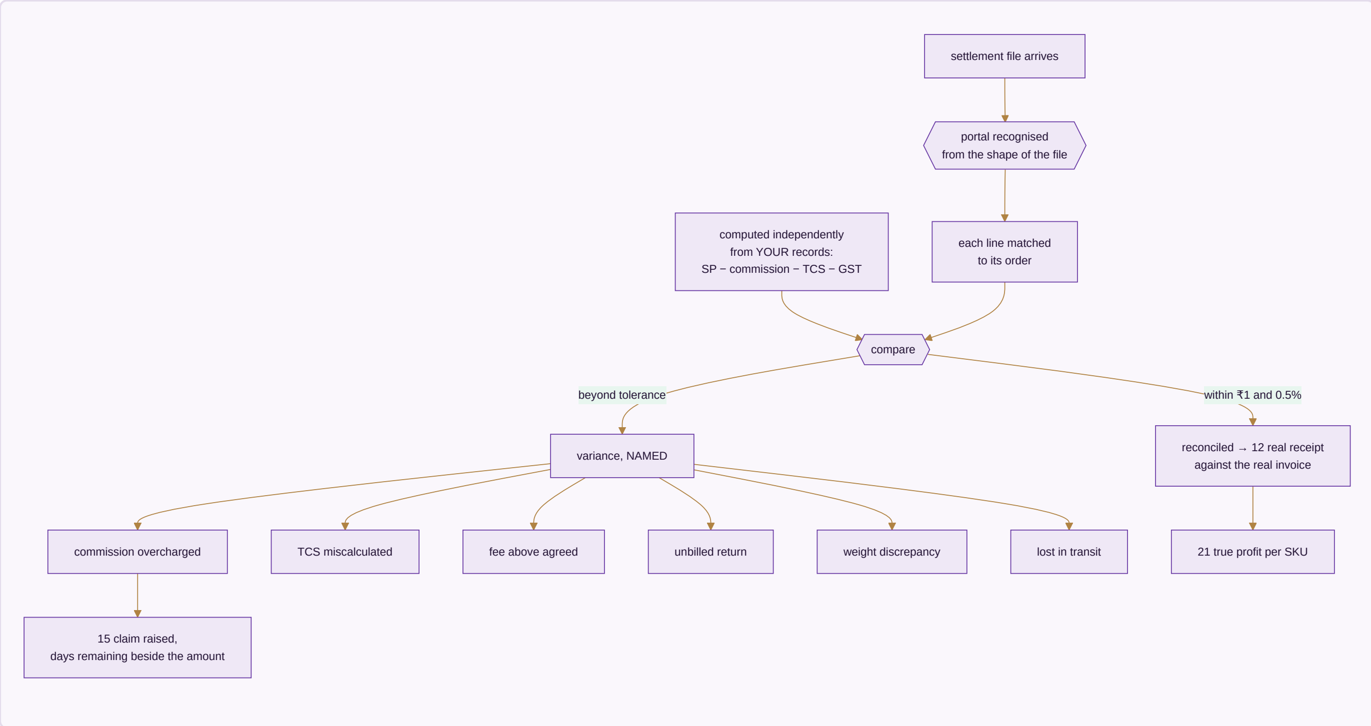
Done when. A twelve-week cash forecast is produced with no manual entry, and a bank statement reconciles with every unmatched line named.

MODULE 14 · SETTLEMENT

Get paid what you are owed — cycle by cycle

What it is. A marketplace does not pay you what the customer paid. It pays selling price minus a commission, minus a collection fee, minus a shipping charge it decided, minus a return it may or may not have handled, minus tax it withheld — and hands you a file with no total you can trust. This module reads that file, works out what each line *should* have been, and puts the two side by side.

How it works



The apps (3)

App	State	What it does
Payout Cycles	SPEC	What each cycle should pay, what actually landed, and on which day — so a late payout is visible the day it is late
Fee & Commission Audit	SPEC	The published rate against the rate actually charged, by category, SKU and tier
TCS & TDS Register	SPEC	Every rupee withheld, matched against the portal's own figures

What it owns. `settlement_cycles` · `fee_audit_lines` · `tcs_tds_register`, reading `marketplace_settlements` from Module 15

The rules that matter

- **The expected figure is computed from your own records, not read back from the file.** The whole point is comparing the file against an answer the file cannot influence.
- **Commission is read from the file, then challenged against the published rate.** A system that assumes the commission is whatever the file says can never detect an overcharge — it has already agreed to it.
- A line is flagged only past **₹1 or 0.5%**, so the variance list holds real problems rather than a thousand one-paise arguments.
- **Every variance has a named kind.** You dispute a weight discrepancy differently from a lost parcel; the name is what makes the claim actionable.
- Days remaining sit beside the rupees at stake. A valid claim that lapses because nobody saw the clock is money given away.

The gate. A real settlement file must reconcile at **98% or better** automatically, and per-SKU profit must land within **₹10** of your own records. Below that the module is not trustworthy enough to run money on — and a settlement tool you cannot trust is worse than none, because it lends confidence to a wrong number.

Reads ← E-commerce/OMS · Accounting & GST · **Writes** → Accounting & GST

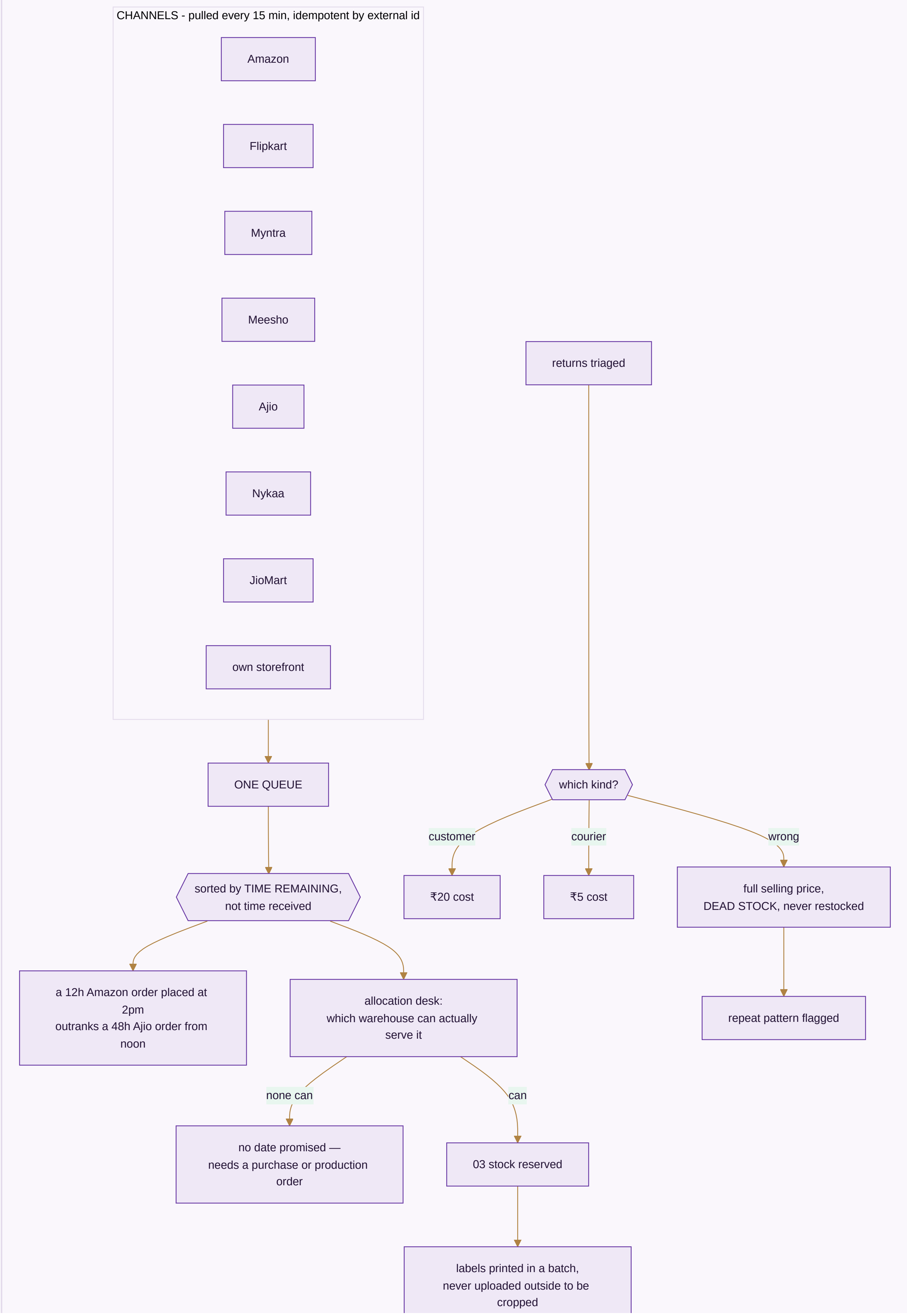
Done when. A real settlement file reconciles at 98% or better and every variance it raises is one you agree is genuinely real.

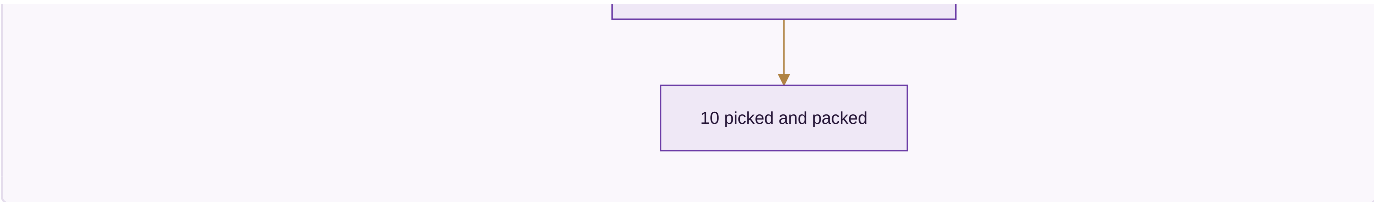
MODULE 15 · E-COMMERCE / OMS

Every marketplace and your own website, one queue

What it is. Stop logging into seven seller panels and your own store admin. Every order lands in one pipeline and one stock number goes back out to all of them — then the money side closes in the same module: what each channel paid, what it kept, what came back, and what it still owes.

How it works


CHANNELS - pulled every 15 min, idempotent by external id
Amazon
Flipkart
Myntra
Meesho
Ajio
Nykaa
JioMart
own storefront
ONE QUEUE
sorted by TIME REMAINING, not time received
a 12h Amazon order placed at 2pm outranks a 48h Ajio order from noon
allocation desk: which warehouse can actually serve it
none can
no date promised — needs a purchase or production order
can
03 stock reserved
labels printed in a batch, never uploaded outside to be cropped
returns triaged
which kind?
customer
₹20 cost
courier
₹5 cost
wrong
full selling price, DEAD STOCK, never restocked
repeat pattern flagged



The apps (11)

App	State	What it does
Marketplace OMS	**BUILT**	Every channel in one order queue, with the right cut-off counting down on each order
Order Management	**BUILT**	One pipeline new to delivered, with the allocation desk and the promise-date engine
Manual Data Check	SPEC	Upload the sheets you already download and read ten cross-checks back, every figure clickable to the transactions behind it
Reconciliation	SPEC	Match every payout to the order line that earned it, and expose the gap
Claims & Disputes	SPEC	Shortfalls, weight disputes and lost parcels filed with evidence, answered before the clock runs out
Returns / RMA	SPEC	Customer, courier and wrong returns kept apart — only one of the three is really your fault
Channels & Storefronts	SPEC	Connect a channel once and it stays in step: catalogue out, price out, stock out, orders in
Labels & Documents	SPEC	The channel's PDF turned into something a packing table can work from, in one batch
Listing & Catalog Manager	SPEC	Bulk create and edit listings across channels, catching listed-but-out-of-stock mismatches
Size / Fit Recommendation AI	SPEC	A fit suggestion from the item's measurements and the return history by size
AR / Virtual Try-On	SPEC	Seeing drape and colour before buying, where a flat photo leaves too much to guess

What it owns. `marketplace_orders_raw` · `marketplace_settlements` · `marketplace_settlement_lines` · `returns` · `claims` · `channels` · `channel_listings`

The rules that matter

- **The queue sorts by time remaining, not time received.** Channels have different dispatch windows; sorting by arrival breaches the tight ones while the loose ones sit safe.
- Orders are pulled on a schedule and are **idempotent by external ID**, so the same order never doubles no matter how often the pull runs.
- **An order no warehouse can serve gets no promised date.** It needs a purchase or a production order, and nothing is gained by giving the customer a date in the meantime.
- **A wrong return is dead stock and is never restocked.** Booking it back into sellable stock is how a business sells the same damaged piece twice and loses the customer permanently.

- **Labels are never uploaded to an outside website to be cropped.** That is a customer's name and address leaving your control for a convenience.
- Commission is read from the settlement file, never assumed — the challenge to it happens in Module 14.

Reads ← Inventory & Catalog · CRM · Sales · Accounting · Logistics · Settlement · **Writes** → Inventory & Catalog · Accounting · Warehouse · Logistics · Settlement

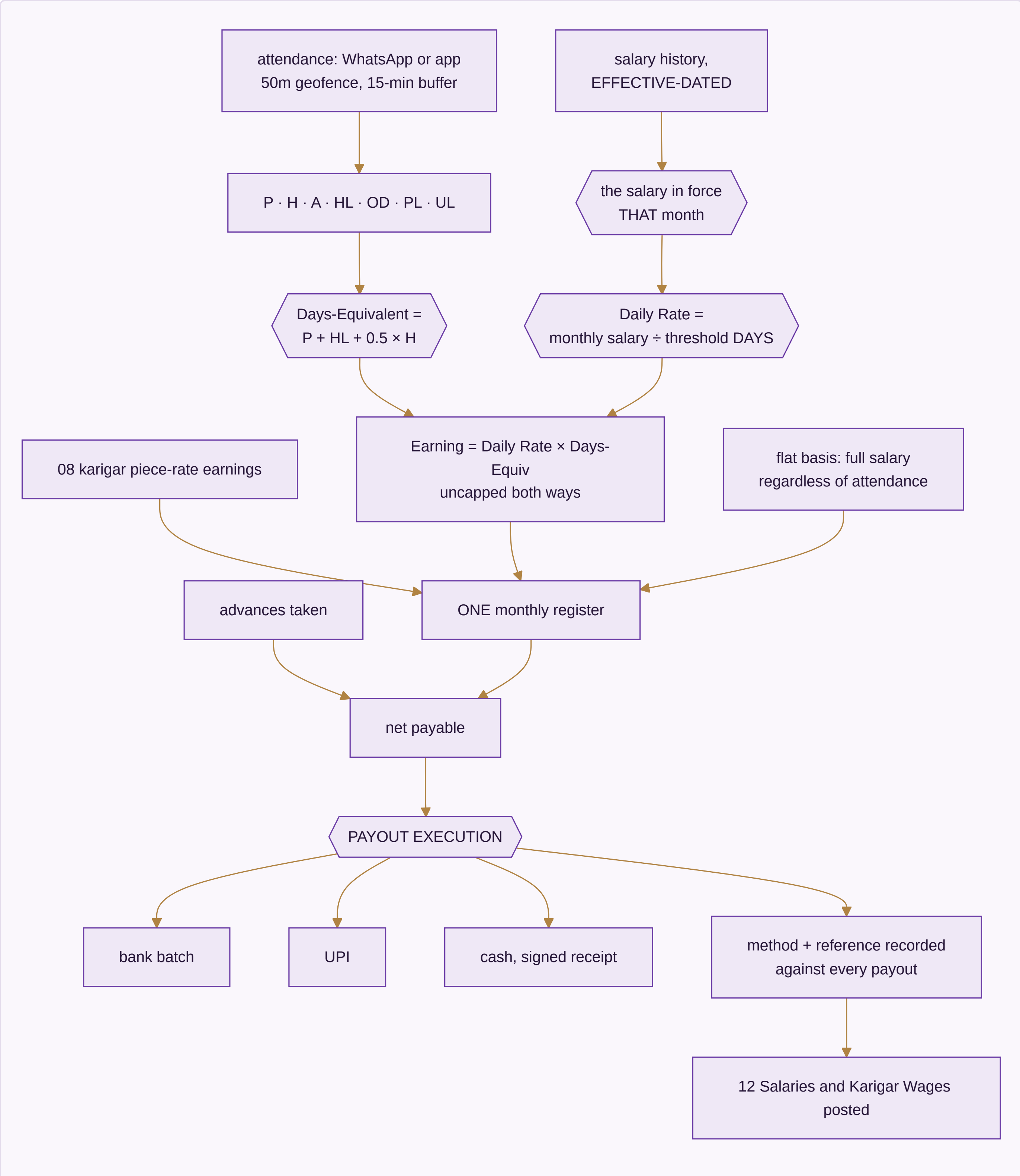
Done when. A full week runs with every channel live, settlements reconciled, and no panel oversells.

MODULE 16 · HR & PAYROLL

Pay people right, on time

What it is. Salaries and output-based earnings in one register, with attendance driving both — whether people are on a monthly wage, an hourly rate, or paid by what they finish. And, unlike the version of this that stops at a calculated figure, it carries through to the money actually leaving the business.

How it works


attendance: WhatsApp or app
50m geofence, 15-min buffer
salary history,
EFFECTIVE-DATED
P · H · A · HL · OD · PL · UL
the salary in force
THAT month
Days-Equivalent =
P + HL + 0.5 × H
Daily Rate =
monthly salary ÷ threshold DAYS
08 karigar piece-rate earnings
Earning = Daily Rate × Days-Equiv
uncapped both ways
flat basis: full salary
regardless of attendance
advances taken
ONE monthly register
net payable
PAYOUT EXECUTION
bank batch
UPI
cash, signed receipt
method + reference recorded
against every payout
12 Salaries and Karigar Wages
posted

The apps (4)

App	State	What it does
Staff & Contractors	SPEC	Attendance, effective-dated salary and output-based earnings in a single register
Time-off & Advances	SPEC	Leave, festival advances, and exactly how they change this month's payout
Appraisal & Hiring	SPEC	Performance reviews and a hiring pipeline that ends in an employee record
Payout Execution	SPEC	Where the calculation becomes money leaving the business, with method and reference on every payout

What it owns. `staff_salary_history` · `attendance` · `eod_reports` · `leave_requests` · `advance_requests` · `payroll_runs` · `payroll_slips` · `karigar_earnings_summary` · `piece_rates` · `task_threshold_rates` · `payout_batches`

The pay rules, exactly

1. Attendance codes are **P · H · A · HL · OD · PL · UL**.
2. **Days-Equivalent = P + HL + 0.5 × H.** A holiday pays a full day.
3. **Daily Rate = the resolved monthly salary ÷ the resolved threshold DAYS** — both effective-dated.
4. **Earning = Daily Rate × Days-Equivalent, uncapped in both directions.** Working beyond the threshold earns beyond the salary; working under it earns under.
5. A flat-basis worker draws full salary regardless of attendance. A piece-rate worker is hours × flat rate with no attendance row at all.
6. Three month-states are distinguished and never conflated: **Not employed**, **No data**, and a real month. Treating the first two as zero is how someone gets paid nothing for a month they worked.

Worked example. Salary ₹15,000 with a 26-day threshold gives a daily rate of ₹576.92. A month of 24 P, 1 HL and 2 H is 24 + 1 + 1.0 = 26.0 days-equivalent, so ₹15,000.00 exactly. The same person on 28.0 days-equivalent earns ₹16,153.85 — it scales past the threshold because the rule is uncapped.

The rule that closes the loop. A salary edit is made **once, with an effective-from date**. The previous row closes automatically, past months keep their old rate, and a future-dated raise activates itself when that month's payroll runs. No historical payroll is ever rewritten.

Reads ← Manufacturing · **Writes** → Accounting & GST

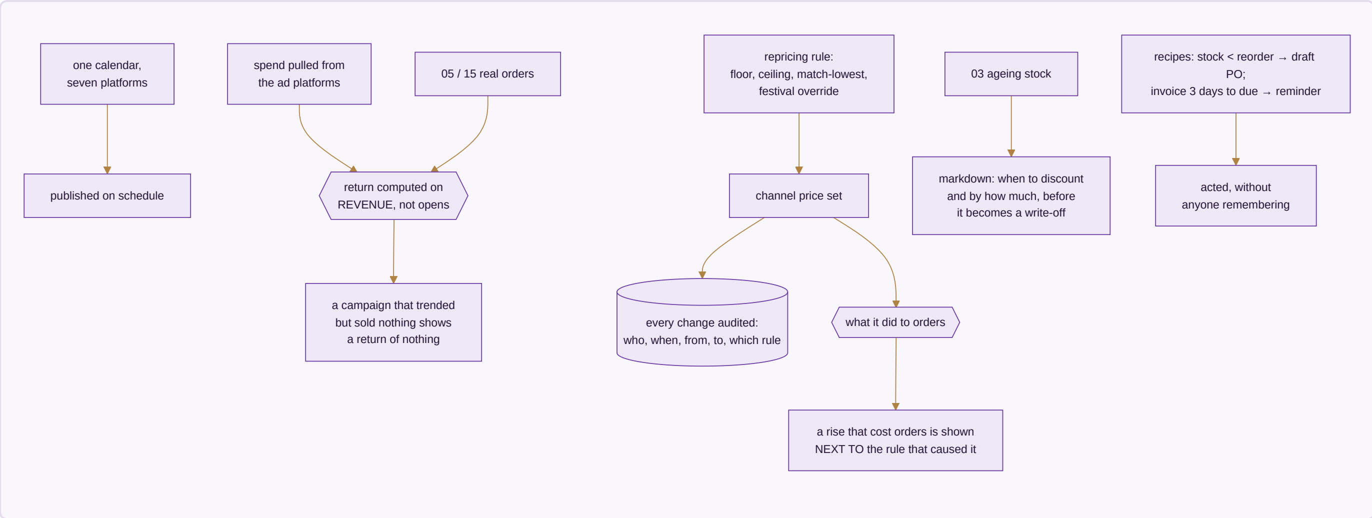
Done when. A full month's payroll runs end to end with zero manual touch, reconciles to the owner's own figures, and every rupee paid has a method and a reference against it.

MODULE 17 · MARKETING

Sell more without discounting

What it is. The demand-generation side of the same order book. Its discipline is that every lever it pulls is judged by what it did to orders and margin — using the same stock number and the same ledger as everything else.

How it works



The apps (7)

App	State	What it does
Social Calendar	SPEC	Plan and publish across every platform from one calendar
Campaigns	SPEC	Campaigns measured on real revenue, not opens
Repricing Engine	SPEC	Rules per channel and SKU, every change audited, and what each one actually did
Automation	SPEC	If this happens, do that — across any module, without writing code
Blog & Pages	SPEC	Articles and landing pages published to your own site with meta and internal links set
Events	SPEC	Trade shows worked as a channel, leads landing straight in CRM
Markdown / Clearance Optimization	SPEC	The repricing engine aimed at ageing stock before it becomes a write-off

What it owns. `content_calendar` · `campaigns` · `influencers` · `asset_library` · `repricing_rules` · `automation_recipes` · `events` · `markdown_plans`

The rules that matter

- **Return is computed on revenue.** Opens, clicks and reach describe attention, not money.
- **A price that rose and took orders down with it is shown as exactly that, beside the rule that raised it.** Pricing is the one lever whose damage is invisible unless you look for it — you simply sell fewer, without being told why.
- Every price change is audited: who, when, from what to what, by which rule. A price is money; an unexplained change to it is the same failure as an unexplained ledger entry.

- **Repricing sets price and only price.** It cannot touch quantity, so no pricing automation can ever oversell by moving a number it had no business touching.
- Discounting ageing stock is a decision with a right time. Too early gives away margin that was available; too late turns a sale into a write-off.

Reads ← Inventory & Catalog · CRM · **Writes** → Sales · E-commerce/OMS

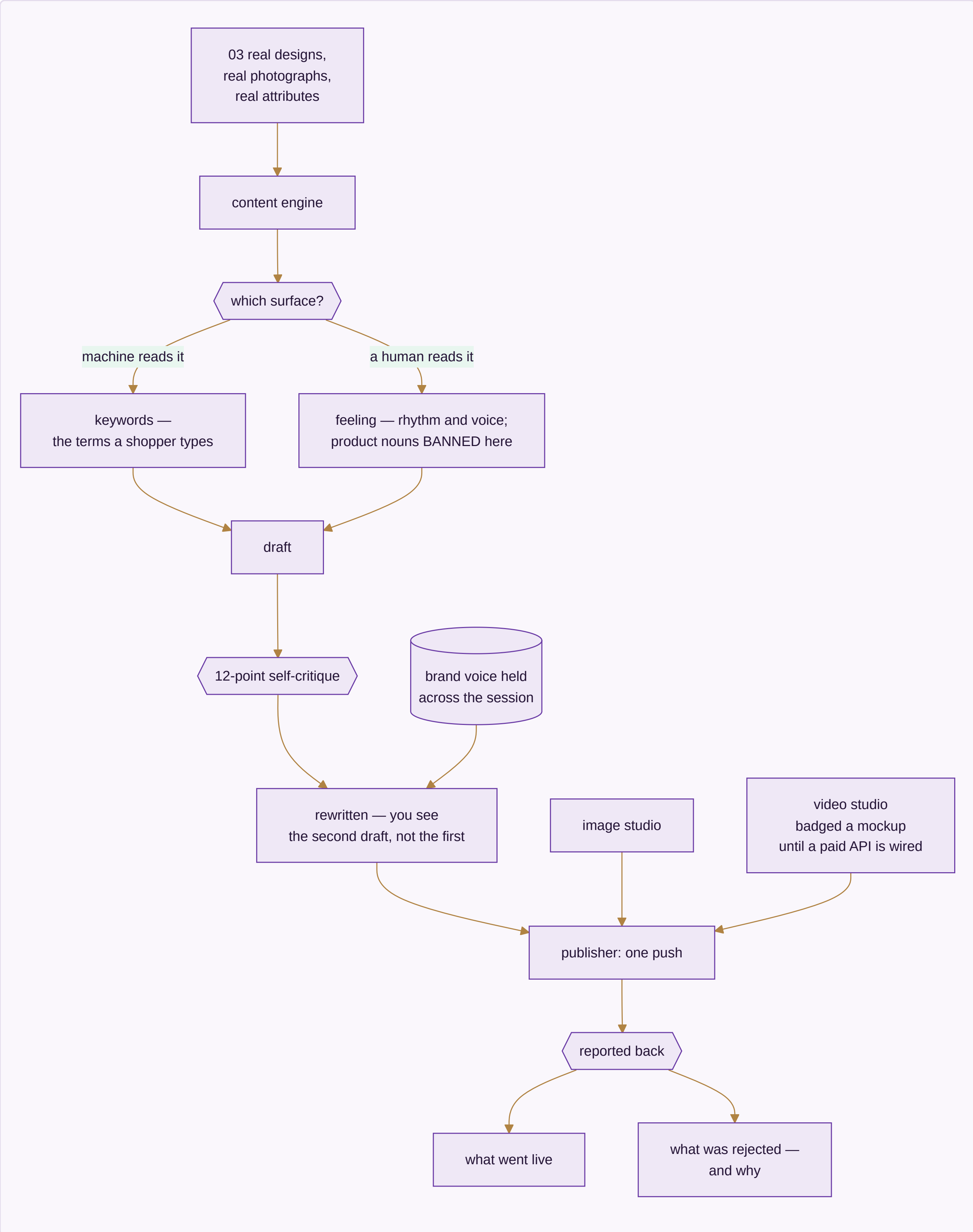
Done when. A month of content publishes on schedule, and a repricing rule can be judged by what it actually did to orders.

MODULE 18 · AI CONTENT ENGINE

Write it, shoot it, cut it — from the catalogue you already have

What it is. The module the platform is named for, deliberately at eighteen rather than first: it produces content *about* the catalogue, so the catalogue has to exist and be correct before there is anything true to write about. Being wired to the same database as the shop floor is what stops it inventing a product that is not real.

How it works


03 real designs, real photographs, real attributes
content engine
which surface?
machine reads it
a human reads it
keywords — the terms a shopper types
feeling — rhythm and voice; product nouns BANNED here
draft
12-point self-critique
brand voice held across the session
rewritten — you see the second draft, not the first
image studio
video studio badged a mockup until a paid API is wired
publisher: one push
reported back
what went live
what was rejected — and why

The apps (5)

App	State	What it does
Content Engine	SPEC	Fourteen stages in your own voice, each written from your own catalogue so the words match the thing
Image Studio	SPEC	Layers, free transform, background removal, channel presets and alt text — a phone photo becomes a channel-compliant product image
Video Studio	SPEC	Text and image to video, reels and ad cuts sized per channel
Design Studio	SPEC	A full design surface exporting at whatever size the channel or printer asks for
Publisher	SPEC	One push everywhere, reporting what went live and what was rejected, with the reason

What it owns. `ai_runs` · `ai_listings` · `ai_design_analytics` · `asset_projects`

The rules that matter

- **Structured data gets keywords; anything a human reads gets feelings.** A back-end field is written for a search algorithm; a caption is written for a person. Writing both the same way is what makes machine-written content read as machine-written.
- **Product nouns are banned from creative surfaces.** The right phrase in a search field is exactly the wrong phrase in a caption, and that bleed is the clearest tell of lazy automation.
- **The engine criticises its own draft before showing it.** A model's first attempt is rarely its best; building the criticism into the pipeline is what clears the bar of publishable rather than demonstrable.
- **Generation stays badged a mockup until a real paid API is wired.** Showing a simulated render as a finished one is exactly the dishonesty this whole platform is built to avoid, so the label is not optional.
- A publish that silently fails on two of six channels leaves you believing you are present where you are not. The report closes that gap.

Reads ← Inventory & Catalog · **Writes** → Marketing · E-commerce/OMS

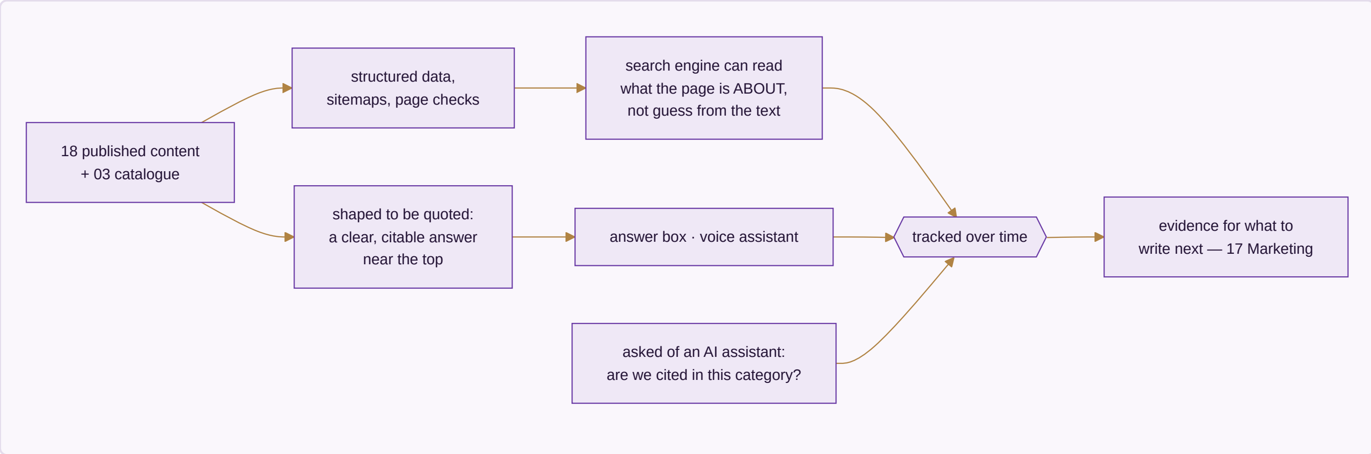
Done when. A listing generates for one design across six platforms in under twenty seconds and publishes, with every rejection explained.

MODULE 19 · SEO, AEO & AIO

Be found by a search box, an answer box, and an AI

What it is. Content exists by the time this module is reached; here it is made findable — by a traditional search engine, by the answer box above the results, and by the AI assistants now answering shopping questions directly instead of sending someone to a results page.

How it works



The apps (3)

App	State	What it does
Technical SEO & Schema	SPEC	Structured data, sitemaps and page-level checks so a search engine can read what a page is about
Answer-Engine Optimization	SPEC	Content shaped to be quoted directly rather than written only to be scrolled
AI-Engine Visibility Tracking	SPEC	Whether this business is actually cited when someone asks an AI a shopping question in this category

What it owns. `seo_audits` · `schema_templates` · `answer_blocks` · `visibility_checks`

The rules that matter

- Structured data is how a page stops being guessed at. Prose alone leaves the engine inferring.
- An answer engine quotes; it does not summarise a page written to be scrolled. Content has to be shaped for the quote to exist.
- **AI visibility is rank tracking aimed at a newer kind of results page.** The discipline is the same — measure over time, act on the trend — and ignoring it means being absent from where a growing share of shopping questions now get answered.

Reads ← Inventory & Catalog · AI Content Engine · **Writes** → Marketing

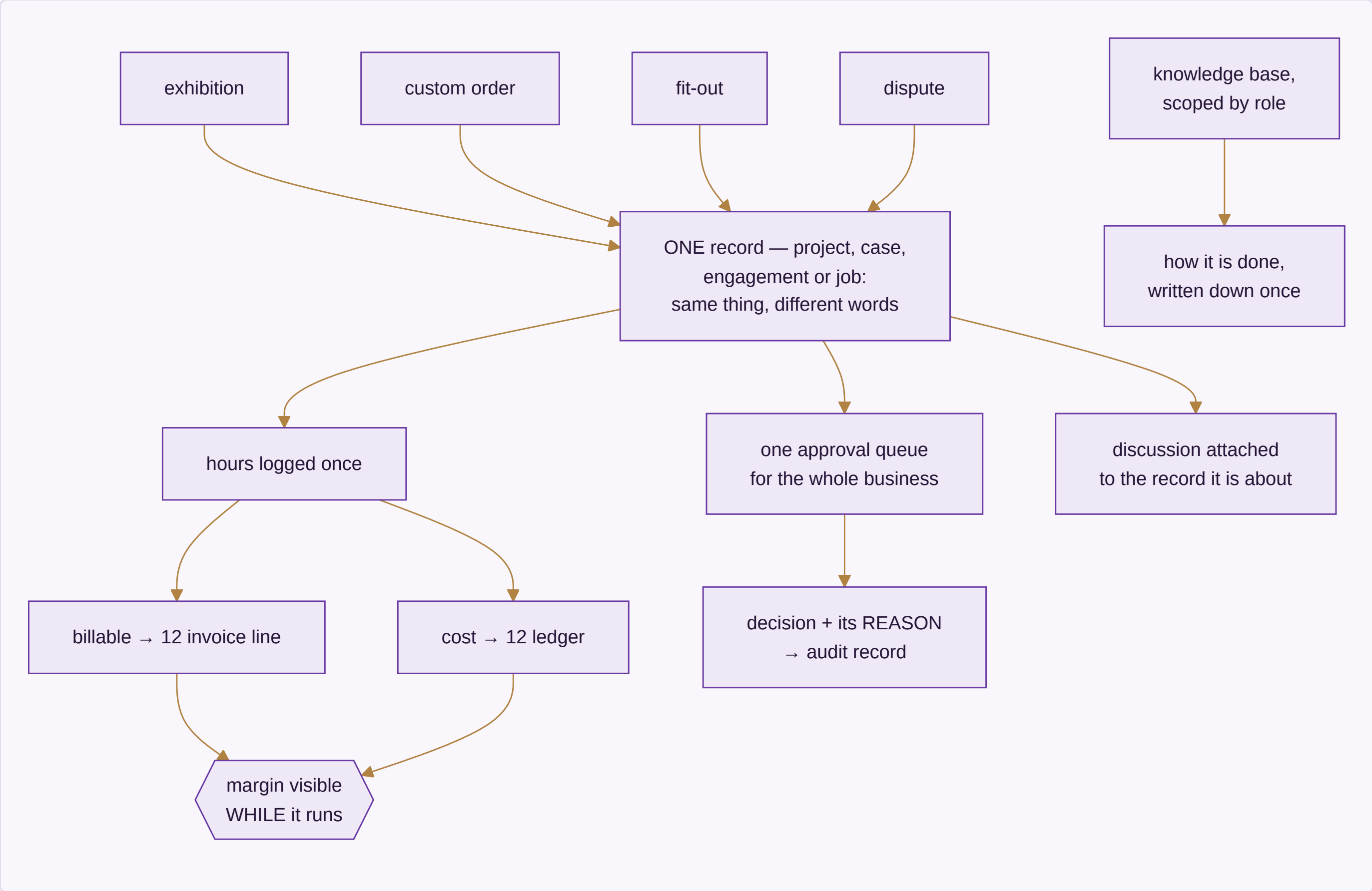
Done when. Every product and content page carries valid structured data, and citation in this category is tracked over time rather than assumed.

MODULE 20 · PROJECTS & COLLABORATION

The work that is not an order — and the talking around it

What it is. Not everything a business does is an order. An exhibition, a custom bulk enquiry that runs six weeks before it becomes an invoice, a godown fit-out, a dispute with a supplier. That work has a deadline, a cost and documents, and it belongs on the same records as everything else.

How it works



The apps (6)

App	State	What it does
Projects & Cases	SPEC	Stages you define, owners, deadlines, documents, billable time and real cost, on one record the ledger can see
Timesheets & Planning	SPEC	Hours against a project or a machine, billable and non-billable kept apart
Approvals	SPEC	One queue for everything waiting on a yes, with the rule that sent it there beside it
Forum	SPEC	Questions and answers that outlive a chat
Discuss	SPEC	Conversation attached to the record it is about
Knowledge Base	SPEC	A role-scoped wiki of standard operating procedures

What it owns. `projects` · `timesheets` · `approvals` · `forum_posts` · `discussions` · `knowledge_base`

The rules that matter

- **Hours are entered once** and become both an invoice line and a cost. Re-keying billable hours from a timesheet into an invoice is exactly where hours get lost and margin quietly leaks.

- **Margin is visible while the work runs.** A margin discovered at the end is a margin you could not steer.
- **Every approval decision carries its reason into the audit record.** A year later, "why did we approve this" has an answer sitting next to the approval, not in someone's memory.
- One approval queue, not one per module — because approvals scattered across modules are approvals that stall unseen.

Reads ← CRM · Sales · HR & Payroll · Inventory & Catalog · **Writes** → Accounting & GST · HR & Payroll · CRM

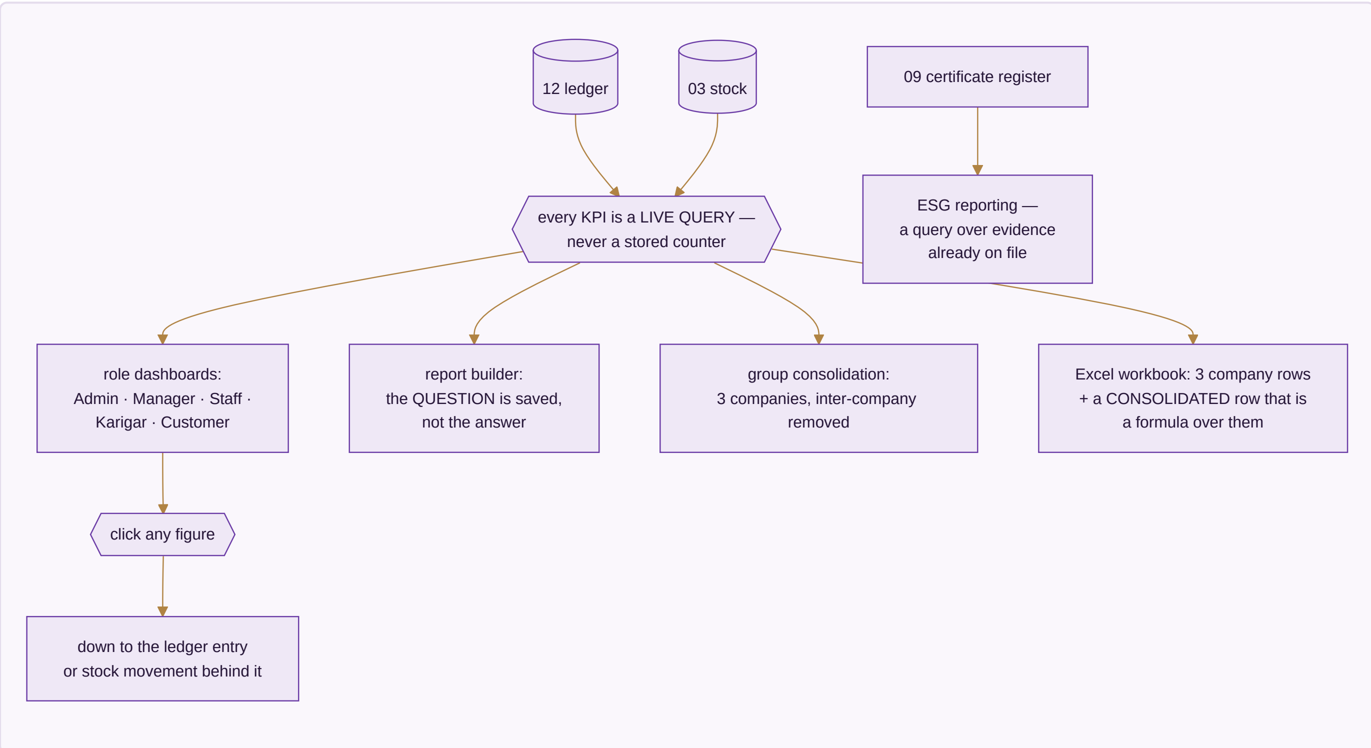
Done when. Billable time on a case becomes an invoice and a cost without being re-keyed once.

MODULE 21 · DASHBOARD & BI

See the whole business without asking anyone

What it is. Every number in the business on one screen, as work happens. It is built last for a reason: it has nothing true to show until the other twenty are producing real records. Two dials govern every screen — which period, and which company.

How it works



The apps (5)

App	State	What it does
CEO Dashboard	**BUILT**	Cash, sales, stock, profit and alerts on one screen, refreshed as work happens
Report Builder	**BUILT**	Drag the fields you want; the question is saved, so next month it answers for next month
Group Consolidation	**BUILT**	Three companies as one set of figures, inter-company entries removed
Excel Dashboard Builder	SPEC	The full workbook, each company a row and the consolidated row a live formula over them
ESG / Sustainability Reporting	SPEC	Water, chemical compliance, waste and packaging, reported from the certificate and audit records already held

What it owns. Nothing. It reads. It stores only saved report definitions and dashboard layouts.

The rules that matter

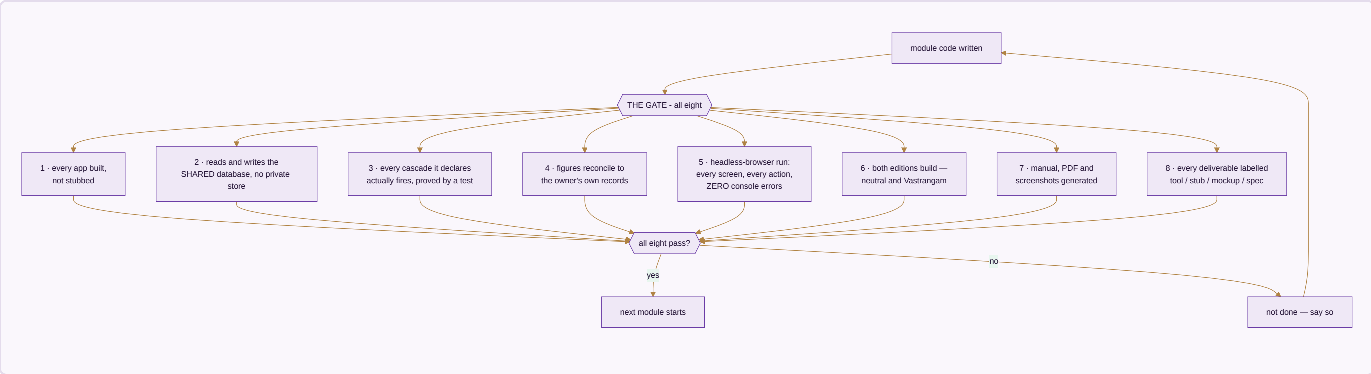
- **Every KPI is a live query over the ledger and stock.** There are no stored counters, because a stored counter is a number that can drift from the truth it claims to summarise.
- **The consolidated row is a formula over the three company rows**, never a separately calculated fourth number — so it foots by construction and an auditor can trust it.
- Group profit removes inter-company sales and purchases. Money the group moved from one pocket to another is not turnover.
- **Any figure clicks down to the record behind it.** A dashboard you cannot interrogate is a dashboard you eventually stop believing.
- The financial year is detected from the data. Nothing is hardcoded.

Reads ← every module · **Writes** → nothing

Done when. Any figure on any screen clicks down to the ledger entry or stock movement behind it, in both editions.

PART III — WHEN IS A MODULE DONE

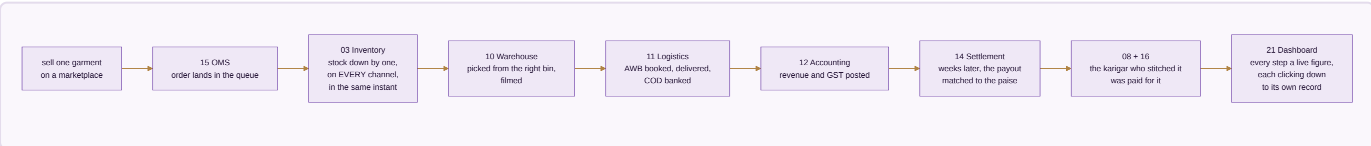
A module is not finished because its code is written. It is finished when all eight of these hold.



Rule 8 is the one that matters most, because it is the one that is easiest to quietly skip. Nothing is called finished that is not. A mockup is labelled a mockup even when a working version would be more impressive to show.

PART IV — THE PROOF

The test of whether this is one system or twenty-one programs sharing a login is a single garment followed end to end. This is re-run at every module boundary.



One transaction. Eight modules. One database.

Everything in this document exists to make that single sentence true. The honest state today is that of the eight modules named in that sentence, sixteen apps are working and the shared database beneath them is written and tested but not yet carrying them — which is the first job of Module 01, and the reason the build order starts where it does.

Every figure in this plan traces to a source: the module and app counts are read directly from the canonical module list the website and every generated document also read, so they cannot drift; the built-versus-spec marks correspond to sixteen apps that pass their own self-tests and a full click-through audit in both editions; and the acceptance figures in Module 08 and the gate in Module 14 are the owner's own numbers, not targets invented here.

Vastrangam Group · Desire to Attire · Surat · Confidential